

ICODA

Illustrated Handbook

C++ reference, complete tutorial, and troubleshooting

ICODA 0.1.0

Built offline from the accepted Markdown documentation

[HANDBOOK.md](#) / [GETTING_STARTED.md](#) / [TUTORIAL.md](#) / [TROUBLESHOOTING.md](#)

Contents

ICODA Handbook	3
1. The ICODEA mental model	3
2. Installation and launch	3
3. Starting a project	4
4. Writing the specification	5
5. Main window tour	17
6. LLM selection	40
7. Architecture workflow	42
8. Implementation workflow	56
9. Build and test gates	66
Worked examples	68
10. Git, persistence, and recovery	69
11. Troubleshooting	70
12. Maintainer guide	73
13. Current limitations	75
14. Command reference	76
15. Glossary	76
Getting started with ICODEA	77
1. Install	77
2. Choose the agent	77
3. Make a new project	77
4. The first step	78
5. From architecture to implementation	78
6. Everyday things	78
Where to go next	79
Full C++ tutorial: Score Clamp from specification to a running program	80
1. Prepare ICODEA and the C++ toolchain	80
2. Create an empty project	80
3. Enter a small, testable specification	81
4. Inspect the skeleton and establish a baseline	81
5. Propose and approve the architecture	82
6. Implement the clamp method in two rounds	84
7. Complete the console program and its output test	88
8. Review the final project in ICODEA	88
9. Complete reference source	90
10. Build, test, run, and inspect the history independently	91
11. Recover from a failed or oversized proposal	91
Troubleshooting	93
Automatic recovery and the Troubleshooting tab	93
Where the log is	93
The launcher refuses to start	93
The window looks frozen	94
The agent call fails	94
A proposal reply is refused before the build	94
The build fails	95
The model is empty, or "no compile_commands.json found"	95
"uncommitted changes in the project"	95
Approve stays grey	95
Saving the specification is refused	96
A window is too tall or the buttons are hidden	96
The demo or a project cannot be reopened	96
Keyboard shortcuts do nothing	96
Source edits cannot be saved	96
Verification (for maintainers)	96

ICODA Handbook

ICODA (Interactive Code Development and Analysis) is a desktop application for developing software in small, reviewable steps. It combines a formal project specification, a derived architecture model, command-line LLM providers, isolated Git worktrees, build and test gates, and an explicit developer approval loop.

This handbook describes the behavior implemented in ICODA 0.1.0. EVOLUTION.md and ICODA_PLAN.md describe the design and its history; this file is the practical guide for using and maintaining the current application.

New users should start with the two-page docs/GETTING_STARTED.md, then docs/TUTORIAL.md; docs/TROUBLESHOOTING.md collects the usual problems and their fixes. The Help menu in ICODA opens all of them.

The examples, tutorial, and screenshots in this handbook use C++. Screenshots are real Tk captures from acceptance fixtures and the Score Clamp tutorial project. Red rectangles identify the areas being discussed.

1. The ICODA mental model

ICODA is built around five ideas:

1. The specification and source code are the project truths. The specification says what the system should do and the source says what it currently does.
2. The architecture view is derived. ICODA parses the project and computes files, entities, calls, type relations, clusters, requirements, test evidence, and implementation status. The diagrams are not a separate model that must be maintained by hand.
3. Development proceeds in phases. A project moves from specification to architecture and then to implementation. The current phase controls which actions are available.
4. Every proposed code change is isolated and verified. The LLM response is applied in .icoda/worktree, built, tested, parsed, and compared with the current project before approval is enabled.
5. The developer owns the decisions. Architecture, implementation approaches, API signature changes, and failed gates remain visible decisions. Automatic approval is deliberately bounded.

ICODA currently supports C++ projects built with CMake presets and Python projects analysed with the standard library ast parser. The GUI is a native Tk desktop application; there is no web interface or background service.

2. Installation and launch

Common prerequisites

- Git
- Python 3.10 or newer with Tkinter
- At least one supported provider CLI if code generation will be used

For C++ projects, also install:

- CMake 3.28 or newer
- Ninja
- Clang and a compatible libclang
- clang-scan-deps when C++20 modules are used
- vcpkg when the project or its Code Profile requires it

On macOS, generated C++ module projects normally need Homebrew LLVM because Apple Clang does not provide the CMake module-scanning configuration used by the generated skeleton:

```
brew install llvm cmake ninja
```

Start ICODA

On macOS or Linux:

```
./icoda.bash
./icoda.bash /absolute/path/to/project
```

On Windows:

```
icoda.cmd
icoda.cmd C:\path\to\project
```

Bootstrap the environment explicitly from the repository's icoda directory. constraints.txt pins every runtime and development dependency used by this checkout; the three launchers (icoda.bash, icoda_python.bash, and icoda.cmd) only select/check Python, Tkinter, tools, and the already prepared environment. They never install or upgrade packages:

```
python3 -m venv .icoda-venv
.icoda-venv/bin/python -m pip install -e '[dev]' -c constraints.txt
./icoda.bash
```

On Windows, use `py -3.12 -m venv .icoda-venv`, then `.icoda-venv\Scripts\python.exe -m pip install -e "[dev]" -c constraints.txt`, then `icoda.cmd`. Repeat the pinned install after `pyproject.toml` or `constraints.txt` changes.

The distributable wheel path is also exercised offline by the test suite. From an environment that already has the pinned build tools and runtime dependencies, build and install it with:

```
.icoda-venv/bin/python -m pip wheel --no-deps --no-build-isolation -w dist .
.icoda-venv/bin/python -m pip install --no-deps dist/icoda-0.1.0-py3-none-any.whl
.icoda-venv/bin/icoda
```

The wheel contains `icoda.py`, both packages, the provider registry, and both JSON schemas. `--no-build-isolation` and `--no-deps` make this path fully offline; prepare dependencies with the pinned bootstrap before disconnecting.

Authenticate a provider

ICODA invokes provider command-line tools; it does not implement their login flows. Authenticate the selected CLI before starting a proposal. The currently enabled providers use:

```
claude          # complete the interactive login
codex login
```

Provider availability and invocation compatibility can be checked without consuming model usage:

```
.icoda-venv/bin/python -m icoda_core.provider_check \
  --output .icoda-test-artifacts/provider-qualification.json
```

3. Starting a project

Create a new project

1. Choose File > New Project... (Ctrl+N; Cmd-N on macOS).
2. Select an empty directory.
3. Complete the Specification window.
4. Press Validate, resolve any reported problems, and press Save (Ctrl+S / Cmd-S).
5. Answer Yes when ICODEA offers to build the skeleton. The build runs in the background and the project is analysed when it passes. Project > Build (Ctrl+B / Cmd-B) repeats this at any time.

At every point the sentence above the buttons in the lower panel names the next action.

Saving the first specification creates only files that do not already exist. It generates either a C++ CMake skeleton or a Python skeleton according to the Code Profile, then moves the project into the architecture phase.

An empty directory receives the C++ defaults. For Score Clamp, the full tutorial included in this PDF, use:

```
Field | Tutorial value
Language / Standard | C++ / 23
C++20 modules | Enabled
Test framework | CTest with standalone C++ test executables
Test runner | ctest --preset debug
Test files | tests/<subsystem>_test.cpp
Source extension | .cppm
Module / Class / Function naming | snake_case / PascalCase / snake_case
Libraries | C++ standard library only
```

The tutorial gives exact specification values, review steps, complete C++ source and tests, and expected output. Its runnable reference project is `docs/examples/score-clamp/`.

Open an existing CMake project

1. Commit or otherwise safeguard the current project first.
2. Build it once with compile-command export enabled.
3. Choose File > Open Project... and select the project root.
4. If analysis reports no compile database, generate `compile_commands.json` as described below and reload.
5. Open Project > Specification... to add or edit the formal specification.

ICODA searches for the newest `compile_commands.json` at the project root, in `build/`, or in a direct child of `build/`. For a CMake project, use:

```
cmake --preset debug
cmake --build --preset debug
```

The preset must set `CMAKE_EXPORT_COMPILE_COMMANDS=ON`. The generated ICODA presets already do this.

An existing CMake project with no ICODA state starts in the implementation phase. Its existing functions are treated as implemented rather than as a new architecture skeleton.

Open an existing Python project

ICODA recursively analyses `.py` files without importing or executing them. A directory without `CMakeLists.txt` starts in the specification phase when it has no existing `.icoda/state.json`. Saving its first specification may add missing Python skeleton files. Commit the project first and inspect the resulting files, especially when the directory is not empty.

What happens on the first proposal

Before asking a provider, ICODA prepares step 0:

- It creates `.icoda/` and the ignored proposal worktree.
- It initializes a Git repository if the project is not already its own repository.
- It builds and analyses the project when a fresh derived model is unavailable.
- It records the current skeleton/model as step 0.
- It creates an icoda step 0: skeleton commit.

Start with a clean, intentional working tree. On a project with no earlier ICODA step, step 0 commits the files that constitute the starting point. Later uncommitted edits are refused until they are handled explicitly.

4. Writing the specification

Open the editor with Project > Specification.... It has six pages and uses JSON Schema validation before save. Hover over a field or its label for an example.

Overview

- Title: a short project name; this is the only field that may not be empty.
- Description: what the program is, who uses it, and the outcome it provides.

Overview

Scope

Use cases

Requirements

Decisions

Code profile

Title	Deterministic Task Engine
Description	A cross-platform library that schedules dependency-aware jobs for interactive and batch applications. Developers need predictable execution, cancellation, diagnostics, and reproducible tests under concurrency and resource pressure.

Validate

Save

Close

Specification overview with a project title and purpose

Overview anchors the rest of the specification in a concrete product and audience.

Scope

- Goals: desired outcomes, one per line.
- Not in scope: features or qualities deliberately excluded.
- Not allowed: forbidden libraries, techniques, dependencies, or implementation choices.
- Done when: observable completion conditions.

Write exclusions as carefully as goals. They prevent a provider from broadening a bounded step into an attractive but unwanted subsystem.

Overview Scope Use cases Requirements Decisions Code profile

Goals	Execute dependency graphs without deadlock Keep scheduling overhead measurable and bounded Make failures and cancellation observable Produce deterministic evidence for every release gate
Not in scope	Distributed execution across machines A graphical workflow editor Persistent cloud queues
Not allowed	Detached worker threads Silent exception suppression Timing-only tests without a deterministic oracle
Done when	Every must requirement has traceable implementation and tests Reference schedules match for randomized dependency graphs Sanitizer, stress, recovery, and platform test suites pass

Validate

Save

Close

Specification scope with goals, exclusions, forbidden choices, and completion criteria

Scope records what success includes, what it excludes, and which implementation shortcuts are forbidden.

Use cases

Use cases describe externally meaningful behavior. Add assigns stable IDs such as UC-1; New clears the form; Remove deletes the selected record. Include the main path and important alternate paths in the details.

For a complex system, use separate records for:

- normal operation;
- invalid input and boundary values;
- resource exhaustion and partial failure;
- cancellation, restart, and recovery;
- persistence and compatibility behavior;
- concurrency, timing, or ordering behavior where relevant.

Overview

Scope

Use cases

Requirements

Decisions

Code profile

id	Use case
UC-1	Submit a dependency graph
UC-2	Cancel queued work
UC-3	Recover from worker failure

UC-1 — editing (New starts a new entry)

Use case

Submit a dependency graph

Details

A client submits jobs and dependencies. The engine validates the graph, rejects cycles with an actionable error, and runs ready jobs exactly once.

Add

New

Remove

Validate

Save

Close

Specification use cases with normal, cancellation, and recovery paths

Use cases are stable, numbered behavioral records rather than a single unstructured prompt.

Requirements

Each requirement receives an R-n ID, a priority, optional use-case references, and details. A requirement should be independently testable. Prefer measurable language such as:

R-4: Cancelling a queued job prevents it from starting. Cancellation completes within 100 ms and does not affect jobs that are already running.

Priorities mean:

- must: the project is unacceptable without it;
- should: important, but not a release blocker in every context;
- could: explicitly optional.

Use-case references must name existing UC-n IDs. Validation rejects unknown references and duplicate IDs.

Overview

Scope

Use cases

Requirements

Decisions

Code profile

id	Requirement
R-1	Execute every ready job exactly once
R-2	Cancellation has bounded latency
R-3	Failures preserve diagnostic evidence
R-4	Sustain the target workload

R-1 — editing (New starts a new entry)

Requirement

Execute every ready job exactly once

Priority

must

Use cases

UC-1

Details

For every acyclic graph, the observed execution set equals the reference topological schedule and every dependency completes before its dependant starts.

Add

New

Remove

Validate

Save

Close

Specification requirements with priority, use-case traceability, and measurable details

Requirements connect measurable behavior to use cases and a release priority.

Decisions

Decisions record choices that should not be reopened in every prompt. Examples include the language, rendering or storage library, threading policy, public API constraints, ownership model, persistence format, and compatibility strategy. Record the rationale when it will help a later proposal preserve the intent rather than only the syntax.

Overview

Scope

Use cases

Requirements

Decisions

Code profile

id	Decision
D-1	Use a bounded work-stealing pool
D-2	Use deterministic model-based tests
D-3	Keep platform APIs behind adapters

D-1 — editing (New starts a new entry)

Decision

Use a bounded work-stealing pool

Why

It supports nested parallel work while keeping worker count and resource ownership explicit.

Add

New

Remove

Validate

Save

Close

Specification decisions with implementation policy and rationale

Decisions preserve architectural intent so later proposals do not repeatedly reopen settled questions.

Code Profile

The Code Profile constrains generated code and supplies build/test conventions. Review at least:

- language and standard;
- C++ module use;
- target platforms;
- test framework and test runner;
- test-file and source-file conventions;
- module, class, and function naming;
- library policy;
- function-size budgets used in prompts and few-line grouping;
- class-size limits (methods and data members);
- project-specific style rules.

Overview Scope Use cases Requirements Decisions Code profile

Language	C++
Standard	23
C++20 modules	☒
Platforms	☒ macOS ☒ Linux ☒ Windows
Test framework	doctest
Test runner	ctest --preset debug --output-on-failure
Test files	tests/<subsystem>_tests.cpp
Source extension	.cppm
Module naming	lower_snake_case
Class naming	PascalCase
Function naming	snake_case
Libraries	vcpkg manifest; single-header libraries vendored under third_party/
Max function lines	30
Style rules	Prefer STL algorithms to hand-written loops; lambdas where they make sense; templates for reuse. Every entity carries a Doxygen comment with @brief and, where a requirement applies, @satisfies. Platform independence: no platform API without a portable wrapper.

Validate

Save

Close

C++ code profile with build, test, naming, library, and style conventions

The Code Profile turns project conventions into explicit generation and verification constraints.

The Code Profile's `max_function_lines` and `hard_max_function_lines` values bound few-line grouping and inform the provider. The promotion rule itself is deliberately fixed: `FUNCTION_LINE_GUIDELINE` is 30 and `FUNCTION_LINE_HARD_MAX` is 50. Other editable profile budgets cover methods and data members. Only the generated build settings (build) are kept in the JSON without an editor field; change those with ICODA closed, then check them by reopening the editor and pressing Validate.

A quality checklist

Before leaving the specification phase, confirm that:

- every goal describes an outcome rather than an implementation activity;
- exclusions and forbidden choices are explicit;
- use cases cover normal, boundary, failure, and recovery paths;
- requirements are testable and include relevant limits;
- high-risk behavior has an independent oracle or reference result;
- important architecture choices and their rationale are recorded;
- completion requires passing tests, not merely compiling;
- the Code Profile matches the actual toolchain and project layout.

The current editor validates structure and references. It does not yet conduct an AI-guided interview, discover missing requirements, or prove that the specification is complete. That review remains a developer responsibility.

Link specification items to code

The Coverage view maps exact `@satisfies` tags to requirements. Requirements use their R-n ID; goals use their position as G-1, G-2, and so on.

C++ example:

```
/// @brief Cancels a job that has not started.
/// @satisfies R-4
bool cancel(JobId id);
```

For multiple requirements, put each exact identifier in its own tag:

```
/// @brief Keep a score in the inclusive range 0 to 100.
/// @satisfies R-1
/// @satisfies R-2
/// @satisfies R-3
static int clamp(int value);
```

Matching is exact; text similarity is not used. Because goal IDs are positional, avoid reordering goals casually after code has been tagged.

5. Main window tour

Menus

File creates, opens, reloads, and remembers projects. Project opens the specification, builds the project, sets the test command, opens the libclang chooser, starts proposals, approves the architecture, undoes the last approved step, or commits manual edits. View opens the analysis surfaces, fits diagrams, and toggles recorded test reachability colours. Help opens Getting started, the Tutorial, this handbook and Troubleshooting, opens the current log file, and shows the version and file locations.

Keyboard shortcuts (Command on macOS, Control elsewhere): N new project, O open project, R reload, E specification, B build, Return propose the next step, Q quit; in the specification editor S saves and W closes.

Source editor

The upper-right pane has Entities and Source Editor tabs. Click a file, class, or function in a diagram, the Mind Map, or the Entities list to open its source. Classes and functions jump to their declaration line, highlighted in yellow. Drag the pane dividers to give the editor more space.

- Open... selects another file inside the current project or candidate worktree.
- Save writes your edits; Reload reads the current disk version. Undo and Redo reverse edits, including a complete Replace All operation.
- Find, Previous, and Next search literal text and wrap around the file. Match case controls case sensitivity. Enter a replacement and use Replace for the highlighted match or All for every match.
- With the source editor focused, Command-S on macOS or Control-S elsewhere saves. Command/Control-F focuses Find, Command-Option-F on macOS or Control-H elsewhere focuses Replace, and Command/Control-Z undoes.

Navigating within the same file keeps unsaved edits. Opening another file, switching projects, closing ICODA, or starting a workflow operation offers Save, Discard, or Cancel if edits are pending. Saving project source refreshes the analysis and leaves the changes uncommitted; use Project > Commit Manual Edits to record them. If a build or generation operation is running, wait for it to finish before saving; your buffer is kept.

When the Call View displays a proposal, its source opens from the candidate worktree. Saving those edits resets the proposal's build and test results; use Rebuild before approving. The path above the editor identifies which project or worktree is being edited.

The editor accepts UTF-8 text files up to 2 MiB, preserves the existing newline convention and byte-order mark, and refuses to overwrite a file that changed on disk after opening. If saving fails after an automatic retry, the buffer remains available and Troubleshooting offers help. Project metadata under .git or .icoda cannot be edited here.

ICODA

Filter: Neighborhood: 0=off Collapse all

File View **Call View** Class View Mind Map Coverage Issues Troubleshooting

Root: main From main Depth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

- cluster .
 - file main.cpp
 - class ScoreClamp
 - method clamp
 - function main

main

LLM

Binary Model

Claude Code (Anthropic)

Entities **Source Editor**

score-clamp: main.cpp

Open... Save Reload Undo Redo

Find Previous Next

Replace Replace All

☒ Match case

```
// Score Clamp: a small C++ example.
class ScoreClamp {
public:
  int clamp(int value) const {
    if (value < 0) return 0;
    if (value > 100) return 100;
    return value;
  }
}
```

Match 1 of 4

Phase specification Request

Implementation queue: inactive Scope Grouping ☐ Auto-approve while gates pass

Write the specification and save it (Project ▶ Specification...). Saving writes the project skeleton and moves the project into the architecture phase.

No proposal

Build: not run Tests: not run Signature changes: none

Approach Delta Signatures Summary Diff Build Tests Prompt Reply

Approach round is inactive outside implementation.

Source saved — refreshing analysis; changes remain uncommitted.

Language: C++

The Source Editor displaying a C++ method and its find and replace controls

The red rectangles identify the Source Editor tab, editing and search controls, and the highlighted source line.

The step panel

The lower panel is driven by the project's phase. One sentence above the buttons says what to do next; it changes with every event (analysis running, proposal ready, signature confirmation needed, approach approved, queue empty, automatic approval paused). Only the buttons that belong to the phase are shown: none in specification; Propose, Approve, Reject..., Adapt..., Approve architecture and Undo last step in architecture; Propose approach, Approve approach, Propose, Approve, Reject..., Adapt... and Undo last step in implementation. Confirm signatures appears only when a proposal changes signatures. More... holds the rare actions Rebuild, Open worktree and Commit manual edits. A grey button is still explained: its tooltip says what it does, when it is available, and why it is grey now.

While a step runs, the buttons are replaced by a moving bar, the current activity (asking the agent for the next step (attempt 1 of 3), building the proposal, ...) and, for agent calls, builds and tests, a Cancel button. Cancel kills the running process; the step ends with cancelled - nothing was recorded and the panel keeps its previous content. Approving, undoing and committing cannot be cancelled because they change the project.

The detail tabs are Approach, Delta, Signatures, Summary, Diff, Build, Tests, Prompt and Reply. The last two show exactly what was sent to the agent for the displayed item and what came back, so that a puzzling proposal can be traced to its cause.

Shared graph controls

The controls above the view tabs affect the reusable diagrams:

- Filter hides nodes that do not match.
- Neighborhood dims nodes more than the chosen graph distance from the selected entity; 0 disables it.
- Collapse all collapses the shared Project -> Cluster -> File -> Class -> Function hierarchy.
- The -, Fit, 100%, and + controls adjust the current diagram.

The filter accepts space-separated terms. Plain text searches names; qualified terms include:

```
name:render
kind:function
status:stub
covered:true
stale:false
cluster:core
namespace:app
edge:calls
```

Terms can be combined. Quoted values are accepted. An invalid or unfinished quote is handled as plain text rather than crashing the filter. covered:true matches the structural recorded test reachability index; it is not a test assurance or runtime execution filter.

Overridden target: describe — 2 remaining

Qualified filters narrow a large graph to the entities relevant to the current question.

Diagram interaction

- Scroll over a diagram to zoom around the pointer.
- Drag with the left or middle mouse button to pan.
- Click + or - in the hierarchy panel to expand or collapse a level.
- Hover over a node for source and status details.
- Click a file or source entity to open it in the upper-right Source Editor at its source line.
- Right-click a node for actions that are valid in the current phase.

Filter:

Neighborhood:

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line

Root: appearance_root

Depth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

appearance_root

Hierarchy — click + / -

- cluster Pinned Application Core
- cluster third_party
- cluster util
- cluster tests
- file appearance.cpp
- file appearance_test.cpp

Phase implementation Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach Delta Signature changes Entity summary Source diff Build Tests

No approach approved for the current target.

Neighborhood mode keeps local context prominent while retaining the shape of the surrounding graph.

The right-click menu can show an entity's introducing step, focus an architecture proposal, override the current implementation target, run recorded tests, pin or rename a cluster, or pin a file to a chosen cluster. Disabled items explain that the action is not valid for the current node or workflow state.

Filter:

Neighborhood: 0=off

Collapse all

File View | Call View | Class View | Mind Map | Coverage | Issues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file config.cppm

file simulation.cppm

file geometry.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach | Delta | Signature changes | Entity summary | Source diff | Build | Tests

No approach approved for the current target.

automatic approval disabled

Language: C++

File hierarchy with a persisted pinned and renamed cluster

25

The hierarchy remains available beside every graph; cluster names and file pins survive reloads.

Views

File View shows project files, dependency clusters, typed relations, and external libraries. Clicking a file lists its entities in the right sidebar. Cluster names and pins are persisted.

Filter:

Neighborhood: 0 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

—

Fit

100%

+

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster src

file app.cppm

file main.cpp

file smoke_test.cpp

struct ScoreClamp

method clamp

function run

function main

external std

used clamp

used max

LLM

Binary

claude

Model

claude-fable-5-1

Claude Code (Anthropic)

Entities

	kind	line

Phase implementation

Request

Max entities 5

Batch size 1

Implementation queue: empty — no unimplemented functions

Scope Queue order

Grouping One entity

Auto-approve while gates pass

Every function in the queue is implemented. Change the Scope to find more work, or extend the specification.

Propose approach

Approve approach

Propose

Approve

Reject...

Adapt...

Undo last step

More...

No proposal

Build: not run

Tests: not run

Signature changes: none

Approach

Delta

Signatures

Summary

Diff

Build

Tests

Prompt

Reply

```
project5: 3 files, 5 entities, 20 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib | leftover proposal worktr Language: C++
```

C++ Score Clamp project in File View with module, entry point, and tests

File View joins dependency structure with the expandable source hierarchy.

Call View shows a function-level graph from a selected root. Use From main, Depth, and callers to change the traversal. Recursion is drawn as a loop; uncertain dynamic calls are dashed and marked ?. A proposal's added and changed entities receive distinct outlines.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: dispatch

From main
Depth: 3
callers

-
Fit
100%
+

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

dispatch

fixed_member

base_virtual_member

LLM
Binary: claude
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

	kind	line

Phase implementation
Request (empty: the agent chooses)
Max entities: 5
Batch size: 1

Overridden target: describe — 2 remaining
Scope: Single entity
Grouping: One entity
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically
Build: not run
Tests: not run
Signature changes: none

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation
libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib
Language: C++

Call View distinguishing a fixed call from an uncertain dynamic call

29

Dashed edges and a question mark preserve uncertainty instead of presenting dynamic dispatch as a false fact.

Class View shows classes and structs with fields and methods. It distinguishes inheritance, composition, aggregation, and usage relations. Callable members include their implementation status.

ICODA

Filter:

Neighborhood: 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

1 class / structs

–

Fit

100%

+

STATUS — gray stub · blue implemented · green tested

struct app::ScoreClamp

0 data · 1 methods

int clamp(int) [tested]

LLM

Binary

claude

Model

claude-fable-5-1

Claude Code (Anthropic)

Entities

	kind	line

Phase implementation

Request

Max entities

Batch size

Implementation queue: empty — no unimplemented functions

Scope

Queue order

Grouping

One entity

☐ Auto-approve while gates pass

Every function in the queue is implemented. Change the Scope to find more work, or extend the specification.

Propose approach

Approve approach

Propose

Approve

Reject...

Adapt...

Undo last step

More...

No proposal

Build: not run

Tests: not run

Signature changes: none

Approach

Delta

Signatures

Summary

Diff

Build

Tests

Prompt

Reply

project5: 3 files, 5 entities, 20 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib | leftover proposal worktr

Language: C++

C++ Class View showing ScoreClamp and its implemented static method

Class View makes member signatures and implementation status reviewable as architecture.

Mind Map is a persistent hierarchy of clusters, files, classes, and functions. Nodes include status, requirement IDs, and the introducing step. Clicking an expandable node changes its persisted expansion state; clicking a node with history also loads that step into the review panel.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Mind map: 73 nodes · 9 visible

-
Fit
100%
+

STATUS — gray stub · blue implemented · green tested

- Cluster · src
Implemented · R-8 · step unknown

- File · main.cpp
Implemented · R-8 · step unknown

+ Function · main
Implemented · R-8 · step unknown

+ Cluster · app
Implemented · R-7, R-8 · step unknown

+ Cluster · core
Implemented · R-1, R-2, R-3 · step unknown

+ Cluster · render
Implemented · R-4 · step unknown

+ Cluster · third_party
Implemented · no requirements · step unknown

+ Cluster · util
Implemented · R-5, R-6 · step unknown

+ Cluster · tests
Unknown · no requirements · step unknown

LLM

Binary
claude
Model
claude-fable-5-1

Claude Code (Anthropic)

Entities

	kind	line

Phase implementation
Request (empty: the agent chooses)
Max entities 5
Batch size 1

Overridden target: describe — 2 remaining
Scope Single entity
Grouping One entity
☐ Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run
Tests: not run
Signature changes: none

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation
libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib
Language: C++

Mind Map showing clusters, requirements, status, and step provenance

33

The Mind Map connects structure to requirements and the step that introduced it.

Coverage has two independent sections:

- specification traceability from exact @satisfies tags;
- recorded test reachability for analysed callables from successful step records and static call edges.

Double-click a callable row to open its source location. The second section is a structural provenance index: it does not inspect assertions, execute tests, verify behavior, or measure statement/branch coverage.

Filter:

Neighborhood: 0=off

Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

Specification coverage: 2/3 goals and requirements covered · 1 UNCOVERED

Coverage requires an exact @satisfies identifier; goals use G-1, G-2, ...

Coverage	ID	Kind	Specification item	Implementing entities
Covered	G-1	Goal	Keep the developer informed	centroid
Covered	R-54	Requirement	Show implementing code	centroid
UNCOVERED	R-55	Requirement	Show newly requested work	No implementing entity

Test coverage: 2/44 callables covered · 42 uncovered

Coverage evidence comes from successful recorded test runs and call reachability.

Coverage	Callable	How covered: tests	Evidence
----------	----------	--------------------	----------

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities Batch size

Overridden target: describe — 2 remaining

Scope Grouping ☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

ApproachDeltaSignature changesEntity summarySource diffBuildTests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Coverage view with covered and uncovered specification items

Requirement traceability and structural recorded test reachability are reported as separate dimensions.

Issues lists advisory code-rule findings such as missing documentation, missing @satisfies, large functions, large classes, too many parameters, direct platform API use, and callables with no structurally reaching recorded test identifier. Double-click a row to open the source.

Filter:

Neighborhood: 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

Rule checks: 159 issues · 2 errors · 157 warnings

Double-click an issue to open its offending entity in the editor.

Severity	Rule	Entity	What to do	Location
Error	function-lines	acceptance::open_native	acceptance::open_native is 57 lines; split it below	main.cpp:2
Error	platform-api	acceptance::open_native	acceptance::open_native calls platform API Create	main.cpp:4
Warning	many-parameter	acceptance::open_native	acceptance::open_native has 6 parameters; group	main.cpp:2
Warning	missing-doxxygen	acceptance::open_native	acceptance::open_native has no Doxygen comment	main.cpp:2
Warning	missing-satisfies	acceptance::open_native	acceptance::open_native has no @satisfies tag; link	main.cpp:2
Warning	missing-test	acceptance::open_native	acceptance::open_native has no successful test case	main.cpp:2
Warning	missing-satisfies	kDefaultWidth	kDefaultWidth has no @satisfies tag; link it to its	src/app/config.cpp
Warning	missing-satisfies	kDefaultHeight	kDefaultHeight has no @satisfies tag; link it to its	src/app/config.cpp
Warning	missing-doxxygen	Config::width	Config::width has no Doxygen comment; document	src/app/config.cpp
Warning	missing-satisfies	Config::width	Config::width has no @satisfies tag; link it to its	src/app/config.cpp
Warning	missing-doxxygen	Config::height	Config::height has no Doxygen comment; document	src/app/config.cpp

LLM

Binary

Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request (empty: the agent chooses)

Max entities

Batch size

Overridden target: describe — 2 remaining

Scope

Grouping

☐ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Issues view with errors and warnings linked to source locations

Issues turn Code Profile rules and missing evidence into a source-oriented review queue.

Status and colours

In normal status mode, callable nodes are gray for stub, blue for implemented, and green for tested. The recorded test reachability colour mode uses green for reached and red for not reached. A red warning marker means the whole derived model or an entity's recorded evidence is stale.

Issue findings are currently advisory except where a specific workflow gate, such as the grouped-step requirement for a structurally reaching recorded test identifier, explicitly enforces them.

The colour mode makes structural recorded test reachability visible directly on a graph.

6. LLM selection

The compact LLM panel contains two editable fields:

- Binary is the executable name or full path.
- Model contains the models configured for the resolved binary, but also accepts a typed model ID.

Filter:
Neighborhood: 0=off

File View
Call View
Class View
Mind Map
Coverage
Issues

-
Fit
100%
+

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster src
cluster app
cluster core
cluster render
cluster third_party
cluster util
cluster tests
file confia.confm

LLM

Binary
codex

Model
gpt-5.6-sol

Codex CLI (OpenAI)

Entities

	kind	line

Phase implementation
Request (empty: the agent chooses)
Max entities
Batch size

Current target: Renderer::draw — 3 remaining
Scope Queue order
Grouping One entity
☐ Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

No proposal

Build: not run
Tests: not run
Signature changes: none

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

No approach approved for the current target.

The selector keeps the executable, provider-specific model, and resolved provider identity together in one compact panel.

Changing the binary updates the model list and remembers the model used for each provider during the session. The selection is stored per project in ignored `.icoda/ui.json` and as the default in the user configuration.

The shipped provider registry contains Claude Code, Codex CLI, Gemini CLI, OpenCode, Aider, GitHub Copilot CLI, and Qwen Code. Only Claude and Codex are currently enabled and marked verified. The other entries, including Gemini, remain hidden from the normal Binary dropdown until their invocation is qualified locally and their enabled/verified fields in `icoda_core/providers.json` are deliberately updated.

A full path still resolves to a provider by executable basename. An unknown binary can be typed, but ICODA cannot invoke it because it has no command template; the field displays that warning directly.

Provider calls are bounded to 30 minutes. ICODA retries rate-limit responses up to three times, following a provider-supplied delay when it can parse one. A malformed proposal is also re-requested up to three times with validation or build feedback.

Provider subprocesses receive the prompt on standard input and cannot edit the checkout: Claude Code is invoked with `--disallowedTools Edit,Write,MultiEdit,NotebookEdit,Bash`, and Codex CLI with `--sandbox read-only`. All commands use argument arrays with `shell=False`; bounded subprocess handling limits time and retained output and kills the process group on timeout or cancellation. ICODA itself applies only the validated structured reply in the isolated Git worktree.

Both response parsers enforce `MAX_RESPONSE_BYTES` (200,000 bytes) before searching for or parsing JSON. Validation refuses lone Unicode surrogates as invalid UTF-8, refuses NUL bytes in candidate paths (a literal NUL also makes JSON invalid), and rejects absolute, parent-traversing, metadata, and build-output paths. `response.apply_changes` resolves every destination against the worktree root and refuses a symlink escape before writing or deleting.

7. Architecture workflow

The architecture phase grows the structural skeleton in small increments. Functions introduced here should remain stubs; behavior is implemented later.

Propose an architecture step

1. Confirm the status bar shows a fresh model and the phase is architecture.
2. Select a provider binary and model.
3. Optionally enter a precise Request. If it is empty, the provider chooses the next step from the specification and current model.
4. Set Max entities to bound the architecture delta.
5. Press Propose or right-click a diagram node and choose Propose the next step here.

ICODA builds a prompt from the specification, Code Profile, derived model, build files, relevant rule issues, earlier rejection reasons, and optional focus. The provider returns a structured response. ICODA applies it in the proposal worktree, builds it, runs tests, reparses it, and computes the architecture delta.

An architecture proposal is rejected automatically if it exceeds the entity budget, changes implementation bodies where only stubs are expected, fails to build, fails to test, cannot be parsed, or violates the response contract. The hard 50-line function limit is a gate, not advice: `_quality_refusal` rejects a proposal that adds or changes a function above `FUNCTION_LINE_HARD_MAX`, and `approve` repeats the same check so a post-review change cannot bypass it. Functions from 31 through 50 lines receive only the 30-line guideline warning.

Filter:

Neighborhood: 0 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster src

file app.cppm

file main.cpp

file smoke_test.cpp

function run

function main

LLM

Binary claud Model claud-fable-5-1

Claude Code (Anthropic)

Entities

	kind	line

Phase architecture

Request

Max entities 5 Batch size 1

Implementation queue: inactive

Scope Queue order Grouping One entity ☐ Auto-approve while gates pass

Step 1 is ready. Read the rationale and the Diff tab, then Approve, Reject... (with a reason) or Adapt... (with constraints).

Propose

Approve

Reject...

Adapt...

Approve architecture

Undo last step

More...

Step 1: Add ScoreClamp architecture (attempt 1)

Build: passed Tests: passed Signature changes: none

Keep the C++ example small and verify each requirement with CTest.

Approach

Delta

Signatures

Summary

Diff

Build

Tests

Prompt

Reply

2 entities added, 0 changed, 0 removed; files: src/app/app.cppm

+ struct app::ScoreClamp

+ method app::ScoreClamp::clamp int clamp(int)

Architecture entity budget: 2 / 5

```
project5: 3 files, 3 entities, 5 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib
```

Language: C++

C++ architecture proposal adding ScoreClamp and its method with passing gates

A live architecture proposal exposes both the structural change and its gate status before approval.

Review a proposal

The lower panel contains:

- the provider rationale and questions;
- Delta: added, removed, changed, and renamed entities;
- Signatures: previous and proposed declarations;
- Summary: editable structured intent for adaptation;
- Diff: the exact candidate patch;
- Build and Tests: captured gate output.

Review the source diff and the resulting diagrams, not only the rationale. Build and test success prove the configured checks passed; they do not prove the change meets the specification.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: main
- Fit 100% +

From main
Depth: 3
callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -
cluster Pinned Application Core
cluster third_party
cluster util
cluster tests
file config.cppm
file simulation.cppm

LLM
Binary: claude
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

Phase architecture
Request (empty: the agent chooses)
Max entities: 5
Batch size: 1

Implementation queue: inactive
Scope: Queue order
Grouping: One entity
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Step 1: Add configured answer (attempt 1)

Build: passed
Tests: passed
Signature changes: none

The specification requires it.

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

2 entities added, 0 changed, 0 removed; files: src/app/app.cppm
+ enum Color
+ enum Level

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Language: C++

Proposal Delta tab listing added entities

Delta summarizes the model-level effect rather than forcing reviewers to infer it from text.

The Summary tab provides structured intent that can be edited and returned as hard adaptation constraints.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: main
From main
Depth: 3
callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

main
load_config
log
Renderer::drawn
Simulation::Simulation
Simulation::run
std
format_level
timestamp
Stack::size
Canvas::Canvas
Canvas::shapes
Simulation::draw_
Simulation::popula
Simulation::repor
to_upper

Hierarchy — click + / -
cluster Pinned Application Core
cluster third_party
cluster util
cluster tests
file config.cppm
file simulation.cppm

LLM
Binary: claud
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

Phase architecture
Request (empty: the agent chooses)
Max entities: 5
Batch size: 1

Implementation queue: inactive
Scope: Queue order
Grouping: One entity
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Step 1: Add configured answer (attempt 1)
Build: passed
Tests: passed
Signature changes: none
The specification requires it.

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

```

diff --git a/src/app/app.cppm b/src/app/app.cppm
--- a/src/app/app.cppm
+++ b/src/app/app.cppm
@@ -22,6 +22,7 @@
     ...
     ...
     ...

```

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation
libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib
Language: C++

Proposal Diff tab with the exact candidate patch

49

The Diff tab remains the authoritative view of what the proposal would actually change.

Decide

- Approve promotes the worktree changes, rebuilds and retests the real project, appends the step record, and creates one Git commit.
- Reject... records a reason. The next proposal receives that reason as feedback.
- Adapt... uses edits made in the Summary tab as hard constraints. If the summary is unchanged, ICODA asks for a semicolon-separated free-text instruction.
- More... > Open worktree opens the proposal checkout for manual inspection or editing.
- More... > Rebuild reruns build, tests, parsing, and delta computation after a manual worktree edit.

Rejection is a recorded decision; its reason becomes feedback for the next attempt.

If an existing entity signature changes, Approve remains disabled until Confirm signatures is pressed. Confirmation applies only to the proposal currently displayed and is not persisted as a general permission.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: main
- Fit 100% +

From main
Depth: 3
callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

- cluster Pinned Application Core
- cluster third_party
- cluster util
- cluster tests
- file config.cppm
- file simulation.cppm

LLM
Binary: claude
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

	kind	line

Phase: architecture
Request (empty: the agent chooses)
Max entities: 5
Batch size: 1

Implementation queue: inactive
Scope: Queue order
Grouping: One entity
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Step 2: Change existing signature (attempt 1)

Build: passed
Tests: passed
Signature changes: 1 — confirmation required

The API needs it.

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

```

centroid
Previous: Point centroid(const std::vector<Point> &)
Proposed: Point centroid(const std::vector<Point> &) /* confirmed change */

```

step 2: proposal ready — approve, reject or adapt

Language: C++

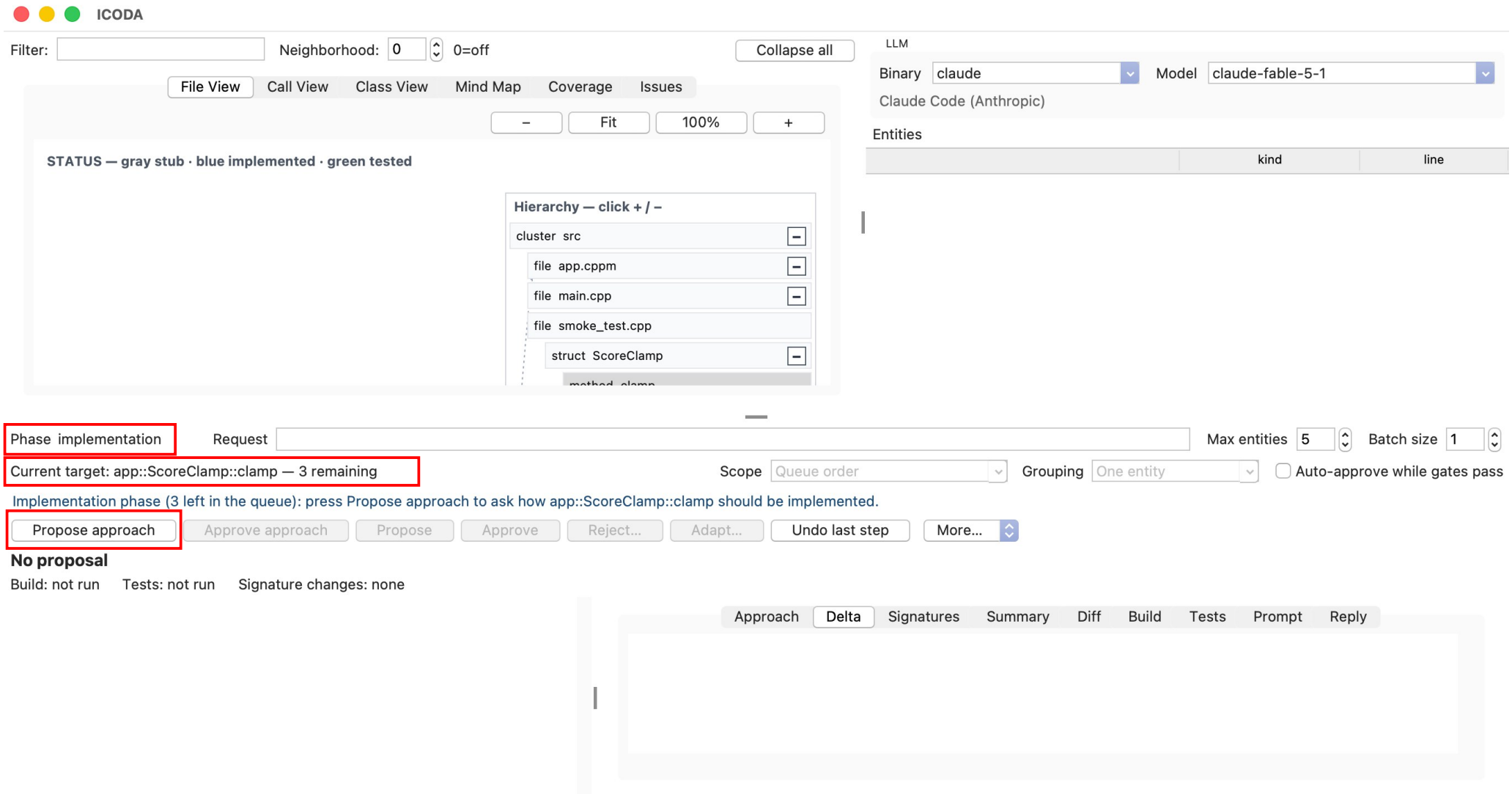
Signature Changes tab requiring explicit confirmation

53

An API change receives its own gate even when build and tests already pass.

Finish architecture

When the skeleton is adequate and no live proposal is waiting for a decision, press Approve architecture. After confirmation, ICODA records the phase transition, constructs the implementation queue, reloads the project, and enables the implementation controls.



```
project5: 3 files, 5 entities, 6 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib | leftover proposal worktree | Language: C++
```

C++ architecture complete with the implementation queue ready

Approving architecture is an explicit phase gate, not an incidental result of approving one proposal.

8. Implementation workflow

Implementation targets functions that are stubs or have no definition. ICODA builds a deterministic, persisted bottom-up queue: callees are implemented before callers. Recursive call cycles are treated as one component with a stable name/signature/file ordering.

Choose the next target

The queue row shows the current target or batch and the number of remaining callables. To override the head, right-click a pending callable and choose Implement this function. The override is persisted and becomes the next two-round implementation target.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: main
- Fit 100% +

From main
Depth: 3
callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

LLM
Binary: claude
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

	kind	line

Phase: implementation
Request (empty: the agent chooses)
Max entities: 5
Batch size: 1

Overridden target: describe — 2 remaining

Scope: Single entity
Grouping: One entity
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Step 1: Add configured answer (attempt 1) — batch: centroid, describe

Build: passed
Tests: failed
Signature changes: none

The specification requires it.

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

Few-line group test coverage (recorded evidence required for every entity):

- centroid
- describe

sample_project: 13 files, 88 entities, 259 relations, 7 clusters, clustering: seeded_label_propagation | libclang: none found

Language: C++

Implementation queue with an explicitly overridden target

A target override changes the next review unit without discarding the deterministic queue.

Scope controls which remaining targets are eligible:

Scope | Meaning

Queue order | Continue through the persisted bottom-up order.

Single entity | Limit the next selection to the current target.

Enclosing class | Bring remaining callables of the current class together.

Enclosing cluster | Bring remaining callables of the current dependency cluster together.

All remaining leaves | Select remaining call-graph leaves before their callers.

Changing scope reorders the relevant queue tail and invalidates an already approved approach.

Batch size is the maximum ordinary queue batch. A later candidate joins only when it has no unimplemented dependency outside the candidate batch. Keep it at 1 for the smallest possible review unit.

Grouping has two modes:

- One entity uses the ordinary queue/batch path. With Batch size greater than one, an ordinary batch can still contain multiple entities; use Batch size 1 for a literal one-entity step.
- Few-line group deliberately combines an adjacent getter/setter family or overload set only when all members are in one file and enclosing class, each fits `max_function_lines`, and the total fits `hard_max_function_lines`. The queue row shows the exact refusal when grouping is unsafe.

Few-line grouping also requires successful recorded test evidence for every member before approval.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: main
- Fit 100% +

From main
Depth: 3
callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

cluster Pinned Application Core
cluster third_party
cluster util
cluster tests
file config.cppm
file simulation.cppm

LLM
Binary: claude
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

	kind	line

Phase implementation
Request (empty: the agent chooses)
Max entities 5
Batch size 2

Current group (2): centroid, describe — 2 remaining

Scope Queue order
Grouping Few-line group
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Step 1: Add configured answer (attempt 1) — batch: centroid, describe

Build: passed
Tests: failed
Signature changes: none

The specification requires it.

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

Few-line group test coverage (recorded evidence required for every entity):

- centroid
- describe

step 2: signature changes confirmed; approval is now available

Language: C++

Few-line group with two adjacent functions and per-member test evidence

59

Grouping is visible as a bounded batch and adds an explicit evidence requirement for every member.

Round 1: approve the approach

1. Press Propose approach.
2. Review the prose plan, expected entities, and expected files in the Approach tab.
3. Press Approve approach, Reject..., or Adapt....

An approved approach is persisted for the current queue batch. Changing the target, scope, batch size, or grouping invalidates it. The approach round does not modify source files.

Filter:
Neighborhood: 0 0=off
Collapse all

File View
Call View
Class View
Mind Map
Coverage
Issues

Root: main
- Fit 100% +

From main
Depth: 3
callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

cluster Pinned Application Core
cluster third_party
cluster util
cluster tests
file config.cppm
file simulation.cppm

LLM
Binary: claude
Model: claude-fable-5-1
Claude Code (Anthropic)

Entities

	kind	line

Phase implementation
Request (empty: the agent chooses)
Max entities: 5
Batch size: 1

Current target: centroid — 1 remaining

Scope: Queue order
Grouping: One entity
Auto-approve while gates pass

Propose approach
Approve approach

Propose
Approve architecture
Approve
Confirm signatures
Reject...
Adapt...
Rebuild
Open worktree
Undo last step
Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not run
Tests: not run
Signature changes: none

Approach
Delta
Signature changes
Entity summary
Source diff
Build
Tests

Awaiting developer approval

Use std::ranges::find, keep the existing signature, and add a focused test; about 8 lines.

Expected entities:

sample_project: 13 files, 88 entities, 259 relations, 4 clusters, clustering: seeded_label_propagation
libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib
Language: C++

Implementation approach awaiting developer approval

The first round presents a prose plan, expected entities, and expected files before any code is generated.

Round 2: approve code and tests

1. Press Propose after approving the approach.
2. Inspect the delta, structured entity summary, source diff, Build output, and Tests output.
3. Confirm signature changes if present.
4. Approve, reject, adapt, or edit/rebuild the worktree.

On approval, ICODA repeats build and tests in the real project before committing. It advances the queue cursor only after all gates pass and the commit path succeeds. The approved approach is then cleared for the next target.

The second round makes source, tests, model delta, and gate results reviewable together.

Automatic approval

Auto-approve while gates pass defaults off. When enabled, it can approve only a live implementation code proposal whose build and tests pass. It never automatically approves:

- architecture steps;
- implementation approaches;
- historical records;
- failed or missing build/test gates;
- unconfirmed signature changes.

After automatically approving code, ICODA proposes the next approach and stops for the developer's decision. If that approach is approved while auto-approve remains enabled, ICODA starts the code round and can approve the green result. When automatic approval cannot continue, the proposal stays as it is and the hint line above the buttons says Auto-approve paused: <reason>. Decide yourself.; the status bar repeats the reason.

Filter:
Neighborhood: 0 0=off
Collapse all

File ViewCall ViewClass ViewMind MapCoverageIssues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

cluster Pinned Application Core

cluster third_party

cluster util

cluster tests

file config.cppm

file simulation.cppm

file geometry.cppm

LLM

Binary

claude

Model

claude-fable-5-1

Claude Code (Anthropic)

Entities

kind

line

Phase implementation

Request (empty: the agent chooses)

Max entities

5

Batch size

1

Overridden target: describe — 2 remaining

Scope

Single entity

Grouping

One entity

☒ Auto-approve while gates pass

Propose approach

Approve approach

Propose

Approve architecture

Approve

Confirm signatures

Reject...

Adapt...

Rebuild

Open worktree

Undo last step

Commit manual edits

Auto-approve stopped — there is no current implementation proposal to approve automatically

Build: not runTests: not runSignature changes: none

Approach

Delta

Signature changes

Entity summary

Source diff

Build

Tests

No approach approved for the current target.

auto-approve stopped: there is no current implementation proposal to approve automatically

Language: C++

Automatic approval enabled but stopped outside a live green code proposal

Automatic approval is deliberately state-aware and reports why it will not proceed.

9. Build and test gates

Build and test results are separate. A successful build cannot compensate for failed, absent, or unconfigured tests.

For C++ projects, the default proposal gates are:

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug
```

The identifier is `steps.CMAKE_PRESET`, currently "debug". Both `gate_commands` and the direct fallback in `_build_commands` configure with `cmake --preset debug` and build with `cmake --build --preset debug`; when a generated `build.sh` or `build.cmd` is used by `_build_commands`, it receives debug build-only. The configured test command remains a separate gate.

For generated projects, `build.sh debug build-only` or `build.cmd debug build-only` provides the separate build-only contract, while running those scripts without `build-only` also invokes CTest for manual use.

For Python projects, the build gate byte-compiles `src/`, and the test gate uses the Code Profile's Test runner. Targeted test identifiers are appended when ICODA can associate tests with the selected functions.

Test selection uses explicit model associations and successful step-log provenance. The Tests tab says whether a focused subset or the full suite ran. Test output is retained with the step; the structural index marks a callable reached only when a successful recorded test identifier can reach it through analysed call edges.

Filter: Neighborhood: 0=off Collapse all

File View **Call View** Class View Mind Map Coverage Issues

Root: main - Fit 100% +

From main Depth: ☐ callers

STATUS — gray stub · blue implemented · green tested · dashed ? uncertain dynamic call

Hierarchy — click + / -

- cluster Pinned Application Core
- cluster third_party
- cluster util
- cluster tests
- file config.cppm
- file simulation.cppm

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line

Phase architecture Request (empty: the agent chooses) Max entities Batch size

Implementation queue: inactive Scope Grouping ☐ Auto-approve while gates pass

Propose approach Approve approach

Propose Approve architecture **Approve** Confirm signatures Reject... Adapt... Rebuild Open worktree Undo last step Commit manual edits

Step 1: Add configured answer (attempt 1) — the proposal tests fail

Build: passed Tests: failed Signature changes: none

The specification requires it.

Approach Delta Signature changes Entity summary Source diff Build **Tests**

Selected tests:

- tests/answer_test.cpp
- app::answer_returns_configured_value

step 2: signature changes confirmed; approval is now available

Language: C++

Failed proposal gate showing the targeted tests that were selected

A failed test gate remains visible with the exact focused tests; build success cannot make the proposal approvable.

Proposal build and test processes have a 15-minute timeout. Displayed output is limited to the final 6,000 characters so a failing process cannot flood the GUI or step log.

The C++ test command defaults to `ctest --preset debug` in `.icoda/state.json`. There is currently no GUI editor for that field. Change `test_command` only with ICODA closed, preserve it as a JSON array of command arguments, and reopen the project.

Worked examples

These examples begin in the repository's `icoda` directory with an authenticated `codex` or `claude` command. Provider prose and titles vary, but the commands, GUI labels, gates, and quoted status text below are deterministic.

Worked example A: new C++ project from first launch to approval

Follow the Full C++ tutorial: Score Clamp from specification to a running program later in this PDF for the complete walkthrough. Its Markdown source is Tutorial, and the runnable reference project is `docs/examples/score-clamp/`.

Score Clamp maps any integer to 0 through 100 and prints scores: 0 42 100 for three demonstration inputs. It uses a C++23 module, a small static method, a thin entry point, and two CTests, with no external C++ libraries.

1. Prepare ICODA's pinned environment and a module-capable C++ toolchain. Create an empty folder through File > New Project....
2. Enter title `ScoreClamp`, two use cases (clamp a score and run the demonstration), and four requirements for low, in-range, and high scores plus console output. Use the tutorial's C++ Code Profile.
3. Validate, save, and build. Inspect `src/app/app.cppm`, `src/main.cpp`, `tests/smoke_test.cpp`, and the presets.
4. Request `app::ScoreClamp` with a static `int clamp(int value)` stub. Keep application behavior unchanged. Inspect the delta and source diff; require passing build and test gates, then approve.
5. Explicitly Approve architecture. Keep one entity per implementation step.
6. Approve a prose approach for `std::clamp(value, 0, 100)` plus nine boundary/extreme cases and idempotence checks. Propose the code, inspect it, and approve after both gates pass.
7. Complete the two rounds for `app::run` and application `main` as well. Add the demo CTest for executable output. Continue until the implementation queue is empty.
8. Run the program and both CTests independently, then inspect Git history and requirement traceability.

The full tutorial supplies exact field values, copyable provider requests, complete final C++ sources, output checks, and recovery instructions. Provider prose may vary; observable behavior is the acceptance criterion.

Worked example B: split a C++ function refused by the 50-line gate

Use a separate disposable C++ project titled `LineGate`. Specify one use case, `Classify` a value, and a requirement to return -1 for negative integers, 0 for zero, and 1 for positive integers. Use the Score Clamp C++23 module/CTest profile and keep the hard maximum at 50 lines.

1. Build the generated skeleton. Request a stub `int app::classify(int value)` in `src/app/app.cppm`, with its Doxygen requirement tag. Approve the architecture proposal after both gates pass, then Approve architecture.
2. Select `classify` as the current target. For this deliberate gate exercise, request 51 source lines from the function signature through its closing brace, using repeated explicit branches. Approve the approach, then propose its code.
3. If a 51-line candidate is returned, ICODA refuses it with a message of this form:

`app::classify` is 51 lines; split it below the hard maximum of 50.

The exact name and line count depend on the returned source. Approve stays disabled. This is proposal-time enforcement by `_quality_refusal`, not an Issues warning. If the provider returns a shorter function instead, the oversized-function refusal has not been exercised.

4. Use `Adapt...`: Replace repeated branches with a small classifier. Keep every function within 50 lines, preserve sign classification, and test negative, zero, positive, minimum, and maximum int values.
5. A complete implementation of this small behavior is:

```
int classify(int value) {
    return (value > 0) - (value < 0);
}
```

For genuinely longer behavior, extract focused private helpers. Do not compress statements onto one line to evade the limit.

6. Require passing build and CTest gates, review any signature changes, and approve. The hard 50-line function limit is checked again before promotion. Complete remaining skeleton targets in their own review rounds.

Worked example C: CMake/C++ with CMAKE_PRESET

Use a new Score Clamp project from example A. ICODA's generated C++ skeleton contains debug and release presets; its proposal gates select steps. `CMAKE_PRESET = "debug"`.

1. Save and build the skeleton. For direct terminal commands, select the same compiler as the GUI. On macOS, set CC and CXX to Homebrew LLVM before the first configure, as shown in the full tutorial.
2. Run the stages independently from the project root:

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug
```

3. Check build/debug/compile_commands.json and reload ICODA. File View shows the module, main, and test files.
4. Propose the ScoreClamp architecture addition. Its worktree uses the same configure/build stages and a separate CTest gate. Inspect Build and Tests before approving.
5. To reproduce only the build stage through the generated helper, run `./build.sh debug build-only` (Windows: `build.cmd debug build-only`), followed by `ctest --preset debug`.
6. The initial skeleton has one smoke test; the completed tutorial has smoke and demo. Inspect the registered tests and require zero failures rather than assuming a fixed test count for every project.

10. Git, persistence, and recovery

Project files

Path | Purpose | Normally committed?

.icoda/specification.json	Formal project specification	Yes
.icoda/state.json	Phase, queue, cursor, approach, scope, grouping, and auto-approve state	Yes
.icoda/steps.jsonl	Append-only development and decision history	Yes
.icoda/layout.json	Cluster names and pins	Yes
.icoda/cache/model.json	Derived analysis cache	No
.icoda/ui.json	Per-project provider and UI choices	No
.icoda/icoda.log	Analysis and GUI diagnostic log	No
.icoda/worktree/	Isolated live proposal checkout	No

The generated `.icoda/.gitignore` ignores `cache/` and `ui.json`; step preparation also ignores `worktree/` and `icoda.log`.

Manual edits

ICODA refuses to start a later proposal while ordinary uncommitted project files exist. Choose Project > Commit Manual Edits to analyse those edits, record them as a manual step, and commit them. If proposal startup detects a dirty tree, the GUI offers this path directly.

Undo

Undo last step creates a Git revert commit for the most recent approved ICODA step. The confirmation names the step that goes and says what happens: the files return to the state before that step, nothing is deleted from the Git history or the step log. It works only when the working tree is clean and HEAD is still that step's commit. ICODA will not rewrite unrelated later history; use Git manually when those conditions no longer hold.

Interrupted proposals

A leftover `.icoda/worktree` is preserved and reported when a project is reopened. Inspect it before proposing again if it contains work worth keeping. Starting a fresh proposal resets or recreates the proposal worktree from the current HEAD.

Corrupt state

Invalid JSON in `.icoda/state.json` is refused rather than silently replaced with defaults. The error is shown in the application and logged, and an existing proposal worktree is preserved. Repair or restore the state file from version control before continuing.

State saves use persistence.`atomic_write_text`: a flushed and synced temporary file beside `state.json` is atomically replaced into place, followed by a best-effort directory sync. If replacement fails, the original `state.json` stays intact. The accepted limitation is narrow: if cleanup is also denied, the replacement error is still reported and one orphaned `.state.json.tmp` file can remain beside that intact original; remove the orphan after correcting permissions.

User configuration

Global defaults and recent projects are stored at:

- macOS: `~/Library/Application Support/ICODA/config.json`
- Linux: `${XDG_CONFIG_HOME:-~/.config}/icoda/config.json`
- Windows: `%APPDATA%\ICODA\config.json`

Source navigation uses the built-in Source Editor. The legacy editor command-template setting is retained in configuration for compatibility; it does not change where diagram clicks open source files.

11. Troubleshooting

The model is empty or stale

For C++, build the project (Project > Build) and confirm a current `compile_commands.json` exists. Open Project > Choose libclang Library..., select a detected candidate and press Apply: the choice is saved and the open project is reloaded, and because every analysis runs in a fresh child process the new library is used at once. The chooser distinguishes the saved active choice from the library the last analysis actually loaded.

If libclang is unavailable or parsing crashes, ICODA keeps the last derived model and marks it stale. Details and the last parsed unit are in `.icoda/icoda.log`.

Detected libclang libraries

- ☐ Automatic detection
- ☒ /opt/homebrew/opt/llvm@22/lib/libclang.dylib (homebrew) [active]
- ☐ /opt/homebrew/opt/llvm/lib/libclang.dylib (homebrew)
- ☐ /Library/Developer/CommandLineTools/usr/lib/libclang.dylib (clt)

Active: /opt/homebrew/opt/llvm@22/lib/libclang.dylib

Currently loaded: libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib

Automatically selected the first detected libclang library.

Apply

Close

The chooser makes toolchain discovery explicit and separates the saved selection from the library last loaded. For Python, syntax-invalid files remain visible with parse errors but contribute no entities or call relations until fixed.

C++ modules do not build

Check all of the following:

- CMake is at least 3.28.
- The selected Clang is module-capable and is accompanied by clang-scan-deps.
- CC and CXX point to matching compiler binaries.
- On macOS, Homebrew LLVM is installed or explicitly selected.
- A build cache copied from another checkout has been removed.

Generated build scripts detect a mismatched CMAKE_HOME_DIRECTORY and replace that preset's stale cache.

A proposal action is disabled

Rest the pointer on the grey button: its tooltip ends with Grey now: <reason>. The reasons are:

- In specification, save a valid specification first (no step buttons are shown).
- In architecture, Propose is available; approach and queue controls are not.
- Approve architecture requires no live proposal.
- In implementation, approve an approach before requesting code.
- Approve requires a usable proposal with passing build and tests.
- Signature changes require Confirm signatures.
- Adapt requires a usable live structured proposal or an unapproved approach.
- Historical records are review-only.
- While a step runs, every button waits; Cancel stops the cancellable parts.

The window does not answer

Slow work runs in the background and shows a moving bar with the activity. If the window itself stops answering for more than five seconds, a watchdog thread writes every thread's stack to .icoda/icoda.log (lines starting with watchdog:); that block identifies the blocked call. docs/TROUBLESHOOTING.md explains what to send.

Provider invocation fails

Run provider qualification, confirm the binary is on PATH, authenticate it interactively, and verify the model ID is accepted by the installed CLI version. A typed full path must still have a basename matching a configured provider command. Provider stderr and the final error appear in the review panel.

Build passes but approval is unavailable

Open the Tests tab. Tests must report passed, not not run. Confirm the test runner exists in the environment and the test command is valid. For a few-line group, every member also needs reachable recorded test evidence.

External edits are not reflected

Return focus to ICODA or choose File > Reload. ICODA snapshots analysed source files and reloads when the window regains focus after a real source change. A changed body can demote a previously tested entity to implemented until tests establish fresh evidence.

Analysis or GUI failure

Read .icoda/icoda.log first. For verification failures, open the newest directory named by .icoda-test-artifacts/LATEST, then inspect summary.txt, the failing check's .log, environment.json, and the captured screenshots.

12. Maintainer guide

Code map

Area | Main files

Desktop shell and File View | `icoda.py`

Larger Tk widgets | `icoda_gui/`

Specification | `icoda_core/specification.py`, `specification.schema.json`, `icoda_gui/spec_editor.py`

Analysis and model | `icoda_core/analysis.py`, `python_analysis.py`, `model.py`, `session.py`

Views and graph behavior | `views.py`, `class_view.py`, `mind_map.py`, `graph_filter.py`, `expansion.py`

Workflow and gates | `steps.py`, `phases.py`, `implementation_queue.py`, `grouping.py`, `auto_approve.py`

History and reachability | `steplog.py`, `coverage_index.py`, `requirement_coverage.py`, `rules.py`

Provider integration | `agent.py`, `providers.json`, `provider_check.py`

Persistence and Git | `persistence.py`, `git.py`

Keep non-GUI decisions in `icoda_core` whenever possible. Tk widgets should render core results and dispatch explicit actions rather than reimplementing queue, reachability, rule, or phase logic.

Required verification after every change

Run the complete project gate:

```
./verify.bash
```

On Windows:

```
verify.cmd
```

The verifier runs all checks even after one fails and retains evidence for each, in this exact order:

1. `git diff --check`
2. Ruff (`python -m ruff check .`)
3. mypy over the application, core, GUI, verifier, GUI acceptance, and real-provider acceptance
4. Python byte compilation
5. local provider qualification
6. real-provider acceptance (`tests/real_provider_acceptance.py`)
7. C++ sample configure, build, and CTest
8. pytest with branch coverage, JUnit XML, and `--cov-fail-under=85`
9. fresh sample-project analysis
10. real-Tk GUI acceptance with validated screenshots

The real-provider stage is fail-closed when no authenticated Codex CLI session is available. A deliberate `./verify.bash --allow-missing-real-provider` run records that stage as SKIP; it does not silently omit it.

Each run creates `.icoda-test-artifacts/<timestamp>/` with command logs, `summary.json`, `summary.txt`, `environment.json`, provider qualification, JUnit, XML/HTML coverage, the sample ICODA log, GUI state, and PNG captures.

`.icoda-test-artifacts/LATEST` names the newest directory.

Do not treat a screenshot's existence as visual verification. The acceptance script checks dimensions, colour variation, visible controls, and graph content; maintainers should also inspect captures affected by a GUI change for clipping, overlap, unreadable labels, incorrect state, and stale output.

ICODA

Filter:

Neighborhood: 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

Specification coverage: 5/5 goals and requirements covered · 0 UNCOVERED

Coverage requires an exact @satisfies identifier; goals use G-1, G-2,

Coverage	ID	Kind	Specification item	Implementing entity
Covered	G-1	Goal	Keep every returned score within 0 through 100	app::ScoreClamp
Covered	R-1	Requirement	Clamp low scores	app::ScoreClamp::clamp
Covered	R-2	Requirement	Preserve valid scores	app::ScoreClamp::clamp
Covered	R-3	Requirement	Clamp high scores	app::ScoreClamp::clamp
Covered	R-4	Requirement	Print the demonstration	app::run, main

Recorded test reachability: 0/3 analysed callables reached · 3 not reached

No reaching recorded test identifiers are available; the overview remains available.

Reachability	Callable	Reaching recorded tests	Evidence
Not reached	app::ScoreClamp::clamp int clamp(int)	No reaching recorded test	—
Not reached	app::run int run()	No reaching recorded test	—
Not reached	main int main()	No reaching recorded test	—

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request

Max entities Batch size

Implementation queue: empty — no unimplemented functions

Scope Grouping ☐ Auto-approve while gates pass

Every function in the queue is implemented. Change the Scope to find more work, or extend the specification.

Propose approach

Approve approach

Propose

Approve

Reject...

Adapt...

Undo last step

More...

No proposal

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signatures

Summary

Diff

Build

Tests

Prompt

Plan

project5: 3 files, 5 entities, 20 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib | leftover proposal worktr Language: C++

Completed C++ lifecycle with the queue empty and coverage evidence visible

The C++ tutorial ends with an empty queue; Coverage separately reports requirement links and recorded test reachability.

Focused acceptance scripts

Run the GUI suite directly when changing views, controls, interaction, or rendering:

```
.icoda-venv/bin/python tests/gui_acceptance.py \
  --project tests/sample_project \
  --output .icoda-test-artifacts/gui-manual
```

Run the deterministic full lifecycle when changing phase, proposal, queue, approval, or persistence behavior. The project directory must be absent or empty:

```
.icoda-venv/bin/python tests/simulation_acceptance.py \
  --project .icoda-test-artifacts/simulation-project \
  --output .icoda-test-artifacts/simulation-manual
```

Run the retained large-project benchmark when changing analysis, layout, expansion, or reachability algorithms:

```
.icoda-venv/bin/python tests/performance_acceptance.py \
  --output .icoda-test-artifacts/performance-manual
```

These GUI scripts require a real display. The standard verifier uses xvfb-run automatically on headless Linux when it is available.

Real-provider acceptance

Real-provider acceptance consumes model usage and is the sixth `verify.bash` stage. Run it directly only when a provider, prompt, response, or end-to-end generation contract needs focused qualification, and always use a new output directory:

```
.icoda-venv/bin/python tests/real_provider_acceptance.py \
  --provider codex \
  --model gpt-5.6-sol \
  --output .icoda-test-artifacts/real-provider-codex-YYYYMMDD-HHMMSS
```

It retains the prompt, provider command, stdout/stderr, parsed response, source diff, build/test results, model facts, step records, Git history, approval, undo, and final clean-tree checks.

Documentation and release evidence

- `docs/GAP_ANALYSIS.md` is the feature-to-evidence inventory.
- `docs/SIMULATION.md` records the complete developer-controlled lifecycle.
- `docs/RELEASE_MATRIX.md` records only platform behavior actually executed.
- `docs/VERIFY_LOG.md` is the detailed chronological verification record.

Do not upgrade a platform or feature status based on inference. Preserve exact commands, outputs, screenshots, and environment metadata that support the claim.

13. Current limitations

The current implementation has the following important boundaries:

- The specification editor validates a supplied formal specification but does not yet guide the developer through an AI-assisted discovery and choice process.
- Specification traceability is shown in the Coverage table; the all-diagram colour mode represents structural recorded test reachability, not specification-tag traceability or verified behavior.
- Most Code Profile and rule-check findings are advisory rather than approval-blocking.
- C++ proposal gates assume the debug CMake build and test presets.
- Only Claude Code and Codex CLI are enabled in the shipped provider registry. Other provider templates are present but intentionally unqualified; the Binary field's tooltip names them.
- The release matrix qualifies Linux with the full verification gate. macOS has a hands-on usage record without a gate run; Windows launcher and platform paths exist but are unmeasured.

- Retained large-project measurements name non-blocking scaling defects in Python parsing, File View layout, and expansion derivation. Consult docs/GAP_ANALYSIS.md before making performance claims.

14. Command reference

The Project > Build menu entry runs the same build gate as a proposal (cmake --preset debug and cmake --build --preset debug for C++, python -m compileall -q src for Python) and reloads the analysis when it passes. Project > Test Command... edits the test command kept in .icoda/state.json (default ctest --preset debug).

```
# Start the GUI
./icoda.bash [project-directory]

# Install contributor dependencies
.icoda-venv/bin/python -m pip install -e '[dev]' -c constraints.txt

# Run the complete verification gate
./verify.bash

# Qualify local provider CLIs without model usage
.icoda-venv/bin/python -m icoda_core.provider_check \
  --output .icoda-test-artifacts/provider-qualification.json

# Analyse a project in the same child-process path used by the GUI
.icoda-venv/bin/python -m icoda_core.session /absolute/path/to/project

# Build and test a generated C++ project
./build.sh debug

# Build a generated C++ project without the script's CTest stage
./build.sh debug build-only
```

Windows equivalents use icoda.cmd, verify.cmd, .icoda-venv\Scripts\python.exe, and build.cmd.

15. Glossary

Approach: the prose-only first round for an implementation target or batch.

Derived model: ICODA's parsed representation of files, entities, relations, status, documentation, and test associations.

Entity: a parsed namespace, type, function, method, field, variable, alias, or related source construct.

Gate: a condition that must pass before approval, especially build, tests, delta validation, phase consistency, signature confirmation, and grouped-step recorded test reachability.

Implementation queue: the persisted bottom-up order of remaining stub or undefined callables.

Proposal: a structured provider response applied and verified in the isolated worktree.

Step record: one JSON Lines history entry describing a phase transition, approach decision, approved/rejected proposal, manual edit, or undo.

USR: a stable entity identifier used by the derived model and implementation queue.

Worktree: .icoda/worktree, the detached Git checkout in which a live proposal is applied and tested before it can affect the project working tree.

Getting started with ICODA

This is the short path: install, make one small project, get the first step from the agent, approve it. It takes about half an hour the first time. The handbook explains every control; this page only says what to do next. Inside ICODA, the sentence above the buttons in the lower panel says the same thing at every moment.

1. Install

You need Python 3.10 or newer with Tkinter, Git, and a coding agent's command-line tool that is installed and logged in: Claude Code (claude) or Codex CLI (codex). For C++ projects you also need CMake 3.28 or newer, Ninja and Clang; on macOS install LLVM with `brew install llvm` because Apple's Clang cannot build C++20 modules with CMake. For Python projects nothing else is needed.

ICODA never installs system packages, requests administrator privileges, or contacts package indexes when it starts. Install missing system prerequisites yourself; when a launcher finds one, it prints the exact command to run and exits when that prerequisite is required. Then prepare the pinned Python environment explicitly from the `icoda` directory of the repository:

```
cd icoda
python3 -m venv .icoda-venv
.icoda-venv/bin/python -m pip install -e '[dev]' -c constraints.txt
./icoda.bash
```

On Windows, use Command Prompt:

```
cd icoda
py -3.12 -m venv .icoda-venv
.icoda-venv\Scripts\python.exe -m pip install -e "[dev]" -c constraints.txt
icoda.cmd
```

`constraints.txt` records the exact versions used to develop ICODA while `pyproject.toml` defines the supported version ranges. Repeat the corresponding `pip install` command after either file changes. The launchers only validate Python, Tkinter, system tools, and this prepared environment before starting ICODA; they do not modify the machine or environment.

For a non-editable wheel installation, first prepare the pinned dependencies above, then use the offline-tested wheel path:

```
.icoda-venv/bin/python -m pip wheel --no-deps --no-build-isolation -w dist .
.icoda-venv/bin/python -m pip install --no-deps dist/icoda-0.1.0-py3-none-any.whl
.icoda-venv/bin/icoda
```

The wheel includes the entry module, packages, provider registry, and schemas. The `--no-build-isolation` and `--no-deps` flags prevent this path from contacting a package index; they assume the pinned bootstrap is complete.

If the window does not open, look at Troubleshooting (`docs/TROUBLESHOOTING.md`); the log file is named there.

2. Choose the agent

In the LLM box on the right, choose the agent's command in Binary (claude or codex; a full path works too) and a Model. The hint under the boxes tells you when ICODA does not know the binary. The agent's tool must already be logged in - ICODA never asks for keys or passwords.

3. Make a new project

File > New Project... (Ctrl+N, Cmd+N on macOS), choose an empty folder. The specification window opens. Fill in:

- Overview: a title and two or three sentences about what the program does.
- Scope: goals (one per line), what is not in scope, what is not allowed (libraries, techniques), and when the project is done.
- Use cases: things a user does with the program, one per entry. Fill in the fields and press Add.
- Requirements: testable statements. Give each a priority and the use cases it belongs to.

- Decisions: choices that are already made, so that the agent does not reopen them.
- Code profile: use C++23 with C++20 modules for the handbook examples; choose the test framework and size limits.

Every field has a tooltip with an example: rest the pointer on its label. A small first project needs one use case, three requirements and no decisions. Press Save (Ctrl+S / Cmd-S). ICODA writes the project skeleton and asks whether to build it now - answer Yes. The build runs in the background; when it is done, the analysis of the skeleton appears in the File View and the project is in the architecture phase.

If you answered No: Project > Build (Ctrl+B / Cmd-B) does the same later. The hint above the buttons reminds you.

4. The first step

Leave the Request field empty (the agent then chooses the most useful step) or write what the next step should do, for example Add the Inventory class with add and remove. Press Propose (Ctrl+Return / Cmd-Return).

The lower panel shows a moving bar and what ICODA is doing: asking the agent, building the proposal in a separate worktree, running the tests, parsing the result. This takes one to three minutes. Cancel stops it; nothing is recorded then.

Provider subprocesses cannot edit the checkout: Codex uses --sandbox read-only, and Claude disables its edit, write, notebook-edit and Bash tools. ICODA validates the structured reply before applying it in the worktree.

MAX_RESPONSE_BYTES is enforced before JSON parsing; lone Unicode surrogates, NUL bytes in candidate paths, unsafe paths, and a response.apply_changes symlink escape are refused.

When the proposal is ready, ICODA switches to the Call View with the new entities marked. Read:

- the rationale on the left;
- Diff - the exact change;
- Delta - which classes and functions are added or changed;
- Build and Tests - the output of the checks.

Then decide:

- Approve takes the change into the project and makes one Git commit.
- Reject... asks for a reason; the agent gets it with the next request.
- Adapt... asks again with your constraints (or with the edits you made in the Summary tab).

If the proposal did not build or its tests failed, the title says so and the tabs show the output. Reject or adapt it, or edit the worktree yourself and choose More... > Rebuild.

A function longer than the hard 50-line function limit is also refused at proposal time. ICODA repeats the same _quality_refusal at approval time, so split the function and propose again; the 30-line value remains an advisory guideline for functions no longer than 50 lines.

5. From architecture to implementation

Repeat step 4 until the skeleton has the classes and functions the specification asks for (they stay stubs). Then press Approve architecture. ICODA builds the implementation queue: every stub function, in a sensible order.

In the implementation phase each function takes two rounds: Propose approach asks the agent how it would implement the current target (prose only); Approve approach accepts that; Propose then asks for the code and its tests. The queue label above the buttons names the current target and how many are left. Auto-approve while gates pass lets ICODA continue on its own while build and tests pass; it pauses at the first failure and tells you why.

6. Everyday things

- File > Reload (Ctrl+R / Cmd-R) analyses the project again, for example after you edited files yourself. ICODA also reloads when the window regains focus after a change.
- Undo last step reverts the last approved step with a new commit; the history stays.
- More... > Commit manual edits records your own edits as a manual step. A proposal starts only from a clean tree, so ICODA offers this itself when needed.
- Project > Test Command... sets the command that every proposal must pass (ctest --preset debug by default for C++; the Code profile's test runner for Python).

- Help > Open Log File opens `.icoda/icoda.log`, the first place to look when something is odd.

Everything ICODA writes lives in the project's `.icoda/` folder: the specification, the derived model, the step log and the proposal worktree. The rest is ordinary Git history.

Where to go next

- Tutorial (`docs/TUTORIAL.md`): the complete C++ Score Clamp example, from specification through reviewed implementation, CTest, and a running executable.
- Handbook (`HANDBOOK.md`): every view, control and file, plus the maintainer material.
- Troubleshooting (`docs/TROUBLESHOOTING.md`): the problems people actually meet, and their fixes.

Full C++ tutorial: Score Clamp from specification to a running program

Build a small C++23 application with ICODA, starting from an empty folder and finishing with tested code, a working executable, and recorded Git history. The program clamps integer scores to the inclusive range 0 to 100. For inputs -5, 42, and 120, it prints:

```
scores: 0 42 100
```

The example uses C++20 modules, CMake, Ninja, and CTest, following ICODA's generated C++ skeleton. It requires no third-party C++ library. Allow 30 to 60 minutes for a first run; provider calls and compilation determine the actual time. Python runs ICODA itself; the application and every test in this tutorial are C++.

The complete reference project is in docs/examples/score-clamp/. Section 9 reproduces all three source files. Follow the new-project workflow first; use the reference to review the provider's result or recover from an overcomplicated proposal. Generated titles and step numbers can vary after retries. Check the named entities, actual source changes, and gate results instead of expecting identical prose.

1. Prepare ICODA and the C++ toolchain

Install Git, Python 3.10 or later with Tkinter, CMake 3.28 or later, Ninja, and a C++23 compiler with C++20 module dependency scanning. A matching LLVM installation provides clang++, clang-scan-deps, and libclang. On macOS, use Homebrew LLVM; Apple's Clang does not provide the module-scanning integration used here.

From the repository's icoda directory, prepare the pinned ICODA environment:

```
python3 -m venv .icoda-venv
.icoda-venv/bin/python -m pip install -e '[dev]' -c constraints.txt
```

On Windows Command Prompt:

```
py -3.12 -m venv .icoda-venv
.icoda-venv\Scripts\python.exe -m pip install -e "[dev]" -c constraints.txt
```

The launchers check this environment; they do not install packages. Authenticate one enabled provider before launching ICODA. Use its installed executable in the Binary field and select a Model; the tutorial does not require a particular model ID.

For terminal builds on macOS, select Homebrew LLVM before the first configure:

```
export CC="$(brew --prefix llvm)/bin/clang"
export CXX="$(brew --prefix llvm)/bin/clang++"
cmake --version
ninja --version
"$CXX" --version
```

On Linux, set CC and CXX to matching Clang executables if needed. On Windows, use an LLVM/MSVC developer command prompt with Ninja and the Windows SDK available. ICODA also selects a compiler for its build gates. Changing compilers after configuring requires a fresh build directory.

2. Create an empty project

From the repository's icoda directory on macOS or Linux:

```
project_dir="$(mktemp -d /tmp/icoda-score-clamp.XXXXXX)"
printf '%s\n' "$project_dir"
./icoda.bash
```

On Windows, create an empty folder such as C:\work\icoda-score-clamp, then launch icoda.cmd. Keep the folder path for the independent build later.

1. Set Binary and Model in the LLM panel.
2. Choose File > New Project... and select the empty folder.

3. The Specification window opens. Keep Auto-approve while gates pass off throughout this tutorial.

3. Enter a small, testable specification

On Overview, enter:

- Title: ScoreClamp.
- Description: A C++ console example that clamps integer scores to 0 through 100 and demonstrates three inputs.

On Scope, use one item per line:

- Goals: Keep every returned score within 0 through 100.
- Not in scope: Interactive input, files, networking, floating-point scores, and configuration.
- Not allowed: Third-party runtime or test libraries; platform-specific APIs.
- Done when: The boundary and extreme-value tests pass, the program prints scores: 0 42 100, and the implementation queue is empty.

On Use cases, enter each record and press Add:

1. Clamp a score: A caller supplies any int and receives its nearest value in the inclusive interval 0 to 100.
2. Run the demonstration: A user runs the program and sees the clamped values for -5, 42, and 120 on one line.

The editor assigns UC-1 and UC-2. On Requirements, add these records in order, all with priority must. Put the exact behavior in Details:

1. R-1, title Clamp low scores, use case UC-1: every negative int returns 0, including the minimum int.
2. R-2, title Preserve valid scores, use case UC-1: values from 0 through 100 are unchanged; clamping twice gives the same result.
3. R-3, title Clamp high scores, use case UC-1: values above 100 return 100, including the maximum int.
4. R-4, title Print the demonstration, use case UC-2: print scores: 0 42 100 followed by a newline and exit with status 0.

On Decisions, add:

1. Use the standard library. Why: `std::clamp` expresses the range rule without overflow-prone arithmetic.
2. Use one application module. Why: Keep the example in module `app` with a thin executable entry point.
3. Use standalone C++ tests through CTest. Why: A nonzero test exit status signals failure without downloading a test library.

On Code profile, use these values:

```
Field | Value
Language / Standard | C++ / 23
C++20 modules | Enabled
Test framework | CTest with standalone C++ test executables
Test runner | ctest --preset debug
Test files | tests/<subsystem>_test.cpp
Source extension | .cppm
Module / Class / Function naming | snake_case / PascalCase / snake_case
Libraries | C++ standard library only; no external dependencies
Max function lines / Hard max function lines | 30 / 50
Press Validate, resolve reported issues, then Save. Accept Build it now?. A new C++ skeleton contains 11 generated files. After a successful build and analysis, the phase is architecture.
```

4. Inspect the skeleton and establish a baseline

The important generated files are:

- CMakeLists.txt: module library, application executable, and CTest smoke-test target.
- CMakePresets.json: debug/release configure, build, and test presets.
- `src/app/app.cppm`: exported module `app`, initially containing `app::run()`.
- `src/main.cpp`: program entry point that calls `app::run()`.
- `tests/smoke_test.cpp`: separate C++ test executable registered as the smoke CTest.
- `build.sh` and `build.cmd`: platform-aware configure/build/test helpers.

The generated `app::run()` returns 0 but does not print the demonstration. A passing initial smoke test checks only this skeleton; it does not establish R-1 through R-4.

In a second terminal, change to the folder selected in section 2 and run:

```
cd /absolute/path/to/your/icoda-score-clamp-project
./build.sh debug
```

On Windows, use `cd /d C:\work\icoda-score-clamp`, then `build.cmd debug`. Expect a successful build and the single smoke test passing. If the build fails, fix the toolchain before proposing code. Verify that `build/debug/compile_commands.json` exists, then use `Project > Build` or `File > Reload`.

Open File View to find the module, main file, and test file. Coverage should not claim that the specification is complete yet.

5. Propose and approve the architecture

Set Max entities to 5. Enter this request:

```
In src/app/app.cppm add app::ScoreClamp with one public static
int clamp(int value) method. Its body must remain a stub returning 0.
Add Doxygen @satisfies tags for R-1, R-2, and R-3.
Keep app::run(), main(), and the smoke test behavior unchanged.
Do not implement clamping, print output, or add dependencies yet.
```

1. Press Propose. ICODA prepares step 0 and creates an isolated proposal worktree.
2. Read Delta. Expect the `ScoreClamp` type and `clamp` method without unrelated public entities.
3. Read Diff. The method is a stub; existing application behavior is unchanged.
4. Require Build: passed and Tests: passed.
5. Press Approve. ICODA repeats the gates in the project, records the change, and commits it.

Use `Reject...` or `Adapt...` if the provider adds extra features. Do not add final behavior assertions in this architecture step: those assertions would correctly fail against the temporary stub.

Press Approve architecture and confirm. The phase becomes implementation. Keep Scope Queue order, Batch size 1, and Grouping One entity. The generated entry points are recorded as architecture callables too: expect to review `app::ScoreClamp::clamp`, `app::run`, and the application `main`, not only the new method. Test-file callables are excluded from the implementation queue.

The red rectangles mark the implementation phase, queue target, and controls used for the next round.

6. Implement the clamp method in two rounds

If another target is current, right-click `app::ScoreClamp::clamp` and choose `Implement this function`. Keep the remaining callables for their own review. Enter this request and press `Propose approach`:

```
Implement only app::ScoreClamp::clamp using std::clamp(value, 0, 100).
Add <algorithm> in the global module fragment before export module app.
Test minimum int, -1, 0, 1, 42, 99, 100, 101, and maximum int.
Also test that clamping an already-clamped result changes nothing.
Use the smoke CTest with a nonzero return on failure.
Keep app::run() and the application main() unchanged.
```

`Read Approach`. The plan should name `src/app/app.cppm` and `tests/smoke_test.cpp`. It should not add interactive input, dependencies, or signature changes. Press `Approve approach`, then `Propose` for the code round.

ICODA

Filter: Neighborhood: 0=off

File View Call View Class View Mind Map Coverage Issues

STATUS — gray stub · blue implemented · green tested

Hierarchy — click + / -

- cluster src
 - file app.cppm
 - file main.cpp
 - file smoke_test.cpp
 - struct ScoreClamp
 - method clamp

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line

Phase implementation Request

Max entities Batch size

Current target: app::ScoreClamp::clamp — 3 remaining Scope Grouping ☐ Auto-approve while gates pass

An approach for app::ScoreClamp::clamp is ready. Read it in the Approach tab, then Approve approach, Reject... or Adapt....

No proposal

Build: not run Tests: not run Signature changes: none

Approach Delta Signatures Summary Diff Build Tests Prompt Reply

Awaiting developer approval

Use std::clamp(value, 0, 100). Test nine boundary/extreme values and idempotence.

Expected entities:

- app::ScoreClamp::clamp

Expected files:

- src/app/app.cppm

project5: 3 files, 5 entities, 6 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib | leftover proposal worktree Language: C++

C++ implementation approach for the Score Clamp method

Inspect the code and tests. Section 9 contains the complete reference. All nine inputs must have independent expected values; computing an expected value by calling the same method would hide bugs. The extreme inputs also guard against overflow-prone range adjustments.

The method should reduce to:

```
static int clamp(int value) {  
    return std::clamp(value, 0, 100);  
}
```

Require both passing gates, then press Approve. The method advances out of the queue. This request does not change its signature. If Confirm signatures appears, inspect the old and new declarations before accepting.

7. Complete the console program and its output test

Return Scope to Queue order if an override selected Single entity. For target app::run, propose and approve this approach, then propose its code:

```
Implement app::run() to print exactly scores: 0 42 100 and a newline,
using ScoreClamp::clamp(-5), ScoreClamp::clamp(42), and ScoreClamp::clamp(120).
Return 0. Include <iostream> in the global module fragment.
Keep all clamp tests. Add a CTest named demo that runs the ScoreClamp
executable and checks its output. Do not add input parsing.
```

Review the body in section 9 and this addition to the generated CMakeLists.txt:

```
add_test(NAME demo COMMAND ScoreClamp)
set_tests_properties(demo PROPERTIES
    PASS_REGULAR_EXPRESSION "scores: 0 42 100"
)
```

The smoke test verifies values and the application service; demo exercises the executable. The CTest output expression checks that the expected text occurs. Also compare the complete output in section 10 to catch extra output. Approve only after both gates pass.

When the queue reaches the application main, complete its approach/code round too. Request a thin entry point that calls app::run() and converts its result to process exit status 0 or 1, as shown in section 9. Retain the demo test. This small step explicitly closes the entry point's architecture status.

Repeat the two rounds for any additional callable introduced by the provider. A literal return 0; is not proof that an architecture callable has completed implementation review. Finish when the queue says Implementation queue: empty - no unimplemented functions (the GUI may use an em dash).

8. Review the final project in ICODA

1. File View: find the module, main, and C++ test sources.
2. Class View: inspect ScoreClamp and its static clamp method.
3. Call View: follow main to app::run and the clamp method.
4. Coverage: check exact @satisfies links for R-1 through R-4; tests alone do not imply traceability.
5. Build and Tests: inspect the evidence recorded for each approved implementation step.
6. Issues: review code-profile findings even when mandatory gates pass.

ICODA

Filter:

Neighborhood: 0=off

Collapse all

File View

Call View

Class View

Mind Map

Coverage

Issues

Specification coverage: 5/5 goals and requirements covered · 0 UNCOVERED

Coverage requires an exact @satisfies identifier; goals use G-1, G-2,

Coverage	ID	Kind	Specification item	Implementing entity
Covered	G-1	Goal	Keep every returned score within 0 through 100	app::ScoreClamp
Covered	R-1	Requirement	Clamp low scores	app::ScoreClamp::clamp
Covered	R-2	Requirement	Preserve valid scores	app::ScoreClamp::clamp
Covered	R-3	Requirement	Clamp high scores	app::ScoreClamp::clamp
Covered	R-4	Requirement	Print the demonstration	app::run, main

Recorded test reachability: 0/3 analysed callables reached · 3 not reached

No reaching recorded test identifiers are available; the overview remains available.

Reachability	Callable	Reaching recorded tests	Evidence
Not reached	app::ScoreClamp::clamp int clamp(int)	No reaching recorded test	—
Not reached	app::run int run()	No reaching recorded test	—
Not reached	main int main()	No reaching recorded test	—

LLM

Binary Model

Claude Code (Anthropic)

Entities

	kind	line
--	------	------

Phase implementation

Request

Max entities Batch size

Implementation queue: empty — no unimplemented functions

Scope Grouping ☐ Auto-approve while gates pass

Every function in the queue is implemented. Change the Scope to find more work, or extend the specification.

Propose approach

Approve approach

Propose

Approve

Reject...

Adapt...

Undo last step

More...

No proposal

Build: not run Tests: not run Signature changes: none

Approach

Delta

Signatures

Summary

Diff

Build

Tests

Prompt

Plan

project5: 3 files, 5 entities, 20 relations, 1 clusters, clustering: seeded_label_propagation | libclang: clang 22.1.4 (LLVM 22) — /opt/homebrew/Cellar/llvm/22.1.4/lib/libclang.dylib | leftover proposal worktr Language: C++

Completed C++ Score Clamp project with an empty implementation queue

The recorded test reachability index projects successful recorded test identifiers through analysed call edges. It does not measure executed lines or branches and does not prove assertion quality. The explicit boundary cases and output check provide behavioral evidence for this example.

In the reference capture, all five specification items (one goal and four requirements) are linked, while recorded test reachability reports 0 of 3 callables. The standalone CTest run did not supply reaching identifiers to that index. This is an evidence-index limitation: inspect the successful smoke/demo output and their C++ assertions directly. Do not describe the red reachability count as either failing CTests or complete runtime coverage.

9. Complete reference source

These are the final versions of all three C++ source files. The generated project supplies the CMake targets; section 7 adds the demo test. A runnable copy including CMake files is in docs/examples/score-clamp/.

Application module: src/app/app.cppm

Keep standard-library includes in the global module fragment, before export module app;.

```
module;
#include <algorithm>
#include <iostream>
export module app;

/// @brief Operations for the Score Clamp example.
export namespace app {

/// @brief Keeps an integer score in the inclusive range 0 to 100.
/// @satisfies G-1
struct ScoreClamp {
    /// @brief Clamp a score without overflow or changing in-range values.
    /// @satisfies R-1
    /// @satisfies R-2
    /// @satisfies R-3
    static int clamp(int value) {
        return std::clamp(value, 0, 100);
    }
};

/// @brief Print three representative scores and return success.
/// @satisfies R-4
int run() {
    std::cout << "scores: " << ScoreClamp::clamp(-5) << ' '
               << ScoreClamp::clamp(42) << ' '
               << ScoreClamp::clamp(120) << '\n';
    return 0;
}

} // namespace app
```

Executable entry point: src/main.cpp

```
/// @file main.cpp
/// @brief Entry point for the Score Clamp demonstration.
import app;

/// @brief Return a conventional process success or failure status.
/// @satisfies R-4
int main() {
    return app::run() == 0 ? 0 : 1;
}
```

Boundary tests: tests/smoke_test.cpp

Explicit exit codes keep the tests effective even when assertions are disabled in a release build.

```
#include <array>
#include <iostream>
#include <limits>
#include <utility>
import app;

/// @brief Check boundaries, extreme integers, and idempotence.
/// @satisfies R-1
/// @satisfies R-2
/// @satisfies R-3
/// @satisfies R-4
int main() {
    const std::array<std::pair<int, int>, 9> cases{{
        {std::numeric_limits<int>::min(), 0}, {-1, 0},
        {0, 0}, {1, 1}, {42, 42}, {99, 99},
        {100, 100}, {101, 100},
        {std::numeric_limits<int>::max(), 100},
    }};
    for (const auto& [input, expected] : cases) {
        const int actual = app::ScoreClamp::clamp(input);
        if (actual != expected ||
            app::ScoreClamp::clamp(actual) != actual) {
            std::cerr << "Unexpected result for " << input << '\n';
            return 1;
        }
    }
    return app::run();
}
```

10. Build, test, run, and inspect the history independently

From your project root, with the compiler selected as in section 1:

```
cmake --preset debug
cmake --build --preset debug
ctest --preset debug
./bin/Debug/ScoreClamp
git log --oneline
git status --short
```

On Windows, the executable is bin\Debug\ScoreClamp.exe. The generated helper is also available: ./build.sh debug or build.cmd debug configures, builds, and runs CTest.

For the reference result, CTest runs smoke and demo and reports:

```
100% tests passed, 0 tests failed out of 2
```

The program prints exactly scores: 0 42 100 plus a newline and returns 0. Check that Git contains the skeleton, architecture, and implementation commits. ICODEA metadata can have pending state changes; ordinary C++ source files should match the last approved step.

To try the supplied reference without replaying the workflow, copy docs/examples/score-clamp/ into an empty folder outside the ICODEA checkout and run the same commands there. Open that copy with File > Open Project... if desired. An existing CMake project opens as an existing implementation project; it does not recreate the new-project decision history.

11. Recover from a failed or oversized proposal

If Build fails, inspect the first meaningful compiler error. Missing clang-scan-deps or unsupported modules usually indicates a mismatched toolchain. A missing std::clamp declaration usually means <algorithm> was omitted or placed in the wrong part of the module.

If Tests fail, retain the failing input and expected value in an Adapt... instruction:

The input 101 must produce 100, but the candidate returned 101.
Fix only the clamp implementation. Keep the nine boundary cases
and idempotence check, and rerun smoke and demo.

Use Reject... for a proposal going in the wrong direction. For a small manual correction, open the proposal worktree, edit the candidate, and use Rebuild before reviewing it again. If ordinary project files were edited, use Commit manual edits before starting another proposal.

ICODA enforces the hard 50-line function limit during proposal checks and again before approval. Split a long function into meaningful helpers; do not remove tests or compress statements to bypass the limit. A provider reply exceeding MAX_RESPONSE_BYTES or violating the path/schema contract is refused before application; this is separate from a compiler or CTest failure.

Completion means all four requirements have been reviewed, the expected output and both tests pass, the implementation queue is empty, and the accepted source is recorded in Git.

Troubleshooting

Automatic recovery and the Troubleshooting tab

ICODA attempts recovery before displaying an operational failure. While it works, the status reads Trying automatic recovery. Provider requests receive one read-only retry. When Codex explicitly reports that its CLI is too old for the selected model, ICODA checks the selected executable's installation and tries its supported updater (Homebrew for a Codex cask, otherwise codex update when supported), then retries the same model and request once. App-bundled executables are never overwritten. Cancel stops the active process.

Failed proposals get one additional corrective request in their existing worktree. ICODA reruns the ordinary build, tests, analysis, and phase checks; only a passing candidate can be approved. Project build/test repairs use an isolated architecture candidate when a committed baseline exists. Existing uncommitted project or candidate edits are preserved. ICODA does not invent an approved implementation approach, reset corrupt workflow state, or commit manual edits as an automatic repair. Analysis failures trigger a rebuild and reanalysis. Other failures receive a CLI investigation when a project and provider are available. Attempts are bounded so persistent failures cannot create an endless loop.

If recovery cannot finish, the Troubleshooting tab opens with a plain-language diagnosis and recovery result. Details reveals the diagnostic evidence. Use Send for a conversation with the selected CLI; the conversation includes the failure and previous messages and permits read-only investigation. You can select another installed Binary/Model if the failing provider cannot respond. Common credential forms are redacted from the troubleshooting context.

Use Open CLI for an interactive session in the affected project or candidate worktree, with the failure context and the provider's normal approvals, when edits or login are needed. Return to ICODA and choose Retry step to run the failed operation and its checks again. Selecting a different project clears the chat.

The cases below are the ones that came up while using ICODA, with what to do. Almost every problem leaves a trace in the log, so start there.

Where the log is

Every project writes `.icoda/icoda.log` in its own folder; Help > Open Log File opens it. Before a project is open, ICODA logs beside its settings file: `~/Library/Application Support/ICODA/icoda.log` on macOS, `%APPDATA%\ICODA\icoda.log` on Windows, `~/config/icoda/icoda.log` on Linux (Help > About ICODA shows both paths). The log has one line per event with a timestamp: analysis start and end, every parsed unit, the steps of a proposal, failures with their traceback, and - when the window stops answering - a dump of every thread's stack (lines beginning with `watchdog:`).

When you report a problem, the last thirty lines of the log say more than a screenshot.

The launcher refuses to start

The Bash launcher stops with `icoda: .icoda-venv is missing`; run these commands from `<icoda directory>`: when the prepared environment does not exist. It stops with `icoda: the ICODA environment is incomplete`; run this command from `<icoda directory>`: when ICODA or one of `clang.cindex`, `jsonschema` and `networkx` cannot be imported. The Windows launcher reports the same two causes as `icoda: .icoda-venv is missing`. and `icoda: the ICODA environment is incomplete..` Launch does not install or repair packages automatically.

Create the environment, then install the checkout and its development dependencies with the versions pinned by `constraints.txt`:

```
python3 -m venv .icoda-venv
.icoda-venv/bin/python -m pip install -e '[dev]' -c constraints.txt
```

On Windows:

```
py -3.12 -m venv .icoda-venv
.icoda-venv\Scripts\python.exe -m pip install -e "[dev]" -c constraints.txt
```

Run the commands from the `icoda` directory, then launch again. The complete procedure and prerequisites are in Getting started and Installation and launch.

The window looks frozen

ICODA does its slow work (analysis, the agent call, builds, tests) in the background and shows a moving bar in the lower panel with what it is doing; the status line at the bottom says the same. A step with an agent call takes one to three minutes, a C++ build of a larger project longer. If the bar moves, wait or press Cancel.

If nothing moves for more than five seconds, the window really is stuck. ICODEA notices this itself and writes the stacks of all threads into the log (watchdog: the window has not answered for N s). Wait a moment - a modal dialog may be hidden behind the main window (look in the Dock or task bar) - then send the watchdog: block from the log together with what you did last.

The agent call fails

The step title says Step N: no usable proposal - ... and the Build, Tests and Reply tabs show the details. The usual causes:

- claude is not on the PATH (or codex): the agent's tool is not installed, or ICODEA was started from an environment that does not see it. Type the full path of the executable into Binary.
- The agent asks for a login: run the tool once in a terminal (claude or codex) and log in there. ICODEA uses that login.
- Rate limit or quota messages: ICODEA waits as long as the message says and tries again, up to three times. If it still fails, wait and press Propose again later.
- The reply is not valid JSON / does not follow the contract: the agent answered in prose. ICODEA sends the validation error back and asks again (three attempts). If every attempt fails, write a more precise Request or choose another model. The Reply tab shows exactly what came back.
- Claude Code returns an error immediately: check that the tool accepts the flags ICODEA uses (claude --help must list -p, --model, --output-format and --disallowedTools; python -m icoda_core.provider_check tests this without spending tokens). ICODEA sends the prompt through standard input, so a very long prompt is not the problem.

Two refusals come from steps.StepRunner._invoke_provider before ICODEA can use a reply:

- Gemini CLI (Google) is disabled; choose an enabled provider: the selected entry exists but its enabled setting is false. Choose Claude Code or Codex CLI, the enabled providers in the shipped registry; changing the binary path does not enable a disabled entry.
- Codex CLI (OpenAI) timed out after 1800 seconds followed by If it asks for a login: run 'codex login': the provider process exceeded PROVIDER_TIMEOUT. Check its terminal output and login, then press Propose again. For Claude Code, run claude and log in; for Codex CLI, run codex login.

python -m icoda_core.provider_check checks installed CLI versions and required flags without making a model request. It does not prove that a login is current. See LLM selection for the provider setup procedure.

A proposal reply is refused before the build

the reply exceeds the 200000-byte size limit means _payload_size exceeded MAX_RESPONSE_BYTES before extract_json searched for JSON. Nothing was applied. Narrow the Request to fewer files or a smaller atomic step and press Propose again; if the provider still ignores the bound, choose another model.

The following messages mean the JSON was found, but response.validate refused text or a candidate path:

- files/0/content: must contain valid UTF-8 text identifies a lone Unicode surrogate. Ask the provider to return valid UTF-8 text and propose again.
- files/0/path: must not contain a NUL byte identifies a NUL in a path. Remove the NUL and use a normal project path.
- files/0/path: must be relative to the project root identifies an absolute POSIX or Windows path; files/0/path: must not leave the project root identifies .; and files/0/path: may not touch git, build output or the .icoda folder identifies protected output or metadata. Request a plain relative source or test path inside the project. The full error starts the JSON object does not match the response schema:.

could not apply the files: candidate path 'linked/escaped.txt' leaves the project root is the response.apply_changes symlink-escape refusal. A lexically relative path resolved through a symlink outside the proposal worktree, so ICODA stopped before writing or deleting it. Remove that candidate path or replace the project symlink with a real in-project directory, then propose again. The safety boundary and review procedure are described in LLM selection and The first step.

The build fails

Project > Build and every proposal run the build gate: `cmake --preset debug` followed by `cmake --build --preset debug` for C++. For Python, ICODA discovers project source roots and runs `python -m compileall -q -x <excluded paths> <source roots>`; it does not assume that source is under `src`. The output is in the Build tab and in the Troubleshooting tab after recovery has been attempted (the full text is in the log).

- CMake 3.28 or higher is required: update CMake.
- clang-scan-deps not found, or errors about modules: the compiler must be able to build C++20 modules. On macOS install LLVM with `brew install llvm`; ICODA and the generated `build.sh` use it automatically. On Linux install Clang 16 or newer together with `clang-scan-deps`.
- The cache was configured for another source directory: delete `build/debug` (the generated `build.sh` does this itself) and build again.
- The build works in the terminal but not in ICODA: ICODA runs the build with the environment it was started with. Start `icoda.bash` from a shell where `cmake`, `ninja` and the compiler are on the `PATH`.

The model is empty, or "no compile_commands.json found"

For a C++ project, ICODA needs the compile database that CMake writes when the project is built (the generated presets switch it on). Press Project > Build; if you built by hand, use `cmake --preset debug` -`DCMAKE_EXPORT_COMPILE_COMMANDS=ON` and reload. ICODA looks at the project root, in `build/` and one level below it, and takes the newest file.

"no libclang found": ICODA needs the `libclang` library of a Clang installation to parse C++. Project > Choose libclang Library... lists what it found; choose one, press Apply, and ICODA reloads the project with it (no restart is needed). If nothing is listed, install LLVM (`brew install llvm` on macOS, `libclang-dev` or `llvm` on Linux). If `clang.cindex` is missing from the prepared environment, repeat the pinned install command from Getting started.

A Python project is analysed without a build, but a file with a syntax error contributes no entities until it is fixed; the file list in the status line reports how many files have parse errors.

"uncommitted changes in the project"

A proposal starts from a clean Git tree, so that Approve and Undo are exact commits. If you edited files yourself, ICODA offers to commit them as a manual step. Answer Yes, or commit them in Git and press Propose again. Files that Git ignores (`build/`, `.icoda/worktree`) do not count.

Approve stays grey

The tooltip on the grey button says why. The common cases: the proposal did not build or its tests did not pass (Build: / Tests: under the title must both say passed); the proposal changes function signatures and Confirm signatures has not been pressed; a historical step from the Mind Map is shown instead of the live proposal (press Propose for a new one).

Tests: not run means no test command ran. For C++ the default is `ctest --preset debug`; Project > Test Command... changes it. For Python, the Test runner in the Code profile (for example `python -m pytest`) is used.

`LineGate.classify` is 51 lines; split it below the hard maximum of 50. shows the exact refusal form. It is a required quality refusal, not an advisory issue. `StepRunner._quality_refusal` calls `rules.promotion_refusal` after the proposal is parsed, and approval calls `_quality_refusal` again so an altered or stale over-limit candidate cannot be promoted. Split the function into focused helpers, keep the original and every new function at 50 source lines or fewer, add tests for the same behaviour, and propose again. The worked procedure is Worked example B.

Saving the specification is refused

The message at the bottom of the specification window names the page and the entry: Use cases UC-1: title is missing, Requirements R-2: unknown use case UC-9. Records are added with Add after filling in the fields on the right; a selected record is edited in place and Add starts a new one.

A window is too tall or the buttons are hidden

Every window is sized to the screen it opens on and its buttons are placed first, so this should not happen anymore. If it does, note the screen resolution and the window, and report it. Error dialogs are shortened to ten lines; the full text is always in the step panel and the log.

The demo or a project cannot be reopened

Look at the log for the last lines before the reopen. If the last line is parsing `<unit>`, the analysis child crashed in libclang on that unit; choose another libclang library or exclude the unit. If the analysis reports leftover proposal worktree preserved at ..., a proposal was interrupted; the next Propose resets the worktree. If `.icoda/state.json` is damaged, ICODA refuses to guess: restore it from Git (`git checkout .icoda/state.json`) when it is committed there, otherwise fix the JSON by hand - the message names the field.

If saving reports an error but `.icoda/state.json` is still intact and a `.state.json.tmp` file remains beside it, `persistence._atomic_write_text` failed before `os.replace` and cleanup could not unlink the temporary file. Correct the directory permissions, confirm `state.json` is the intact version you want, and remove only the orphaned `.state.json.tmp` file. Do not replace the good state file with the orphan. See Corrupt state for the recovery procedure.

Keyboard shortcuts do nothing

The shortcuts use the Command key on macOS (Cmd-N, Cmd-O, Cmd-R, Cmd-E, Cmd-B, Cmd-Return, Cmd-S in the source or specification editor) and Control elsewhere. They work when the ICODA window has the focus; inside a text box, Cmd-Return / Ctrl+Return still proposes.

Source edits cannot be saved

The Source Editor keeps the buffer if saving fails and retries the file operation once before showing the problem in Troubleshooting. If the file changed on disk, copy any edits you need to keep before using Reload to inspect the other version. ICODA will not overwrite that version automatically.

Wait for a running build or generation operation to finish before saving. Candidate worktree edits reset the proposal's build and test results; run Rebuild before approval. Project edits stay uncommitted until you use Project > Commit Manual Edits. The built-in editor opens UTF-8 text files up to 2 MiB; use an external editor for other encodings or larger files.

Verification (for maintainers)

`./verify.bash` runs the whole gate and writes its evidence under `.icoda-test-artifacts/`; the newest run is named in LATEST. Read `summary.txt` first, then the failing check's `.log`. The handbook's maintainer guide explains the gate.

FAIL real-provider: missing Codex credential: run 'codex login' means codex login status found no current session; an expired login has the same result. Authenticate with codex login and rerun the gate. If a release is deliberately qualified without that external check, run `./verify.bash --allow-missing-real-provider`: the real-provider stage remains in `summary.txt` and `summary.json` with outcome SKIP instead of being omitted or reported as passed. See Required verification after every change for the full procedure.